

close(2) - Linux man page

Name

close - close a file descriptor

Synopsis

```
#include <unistd.h>
int close(int fd);
```

Description

close() closes a file descriptor, so that it no longer refers to any file and may be reused. Any record locks (see [*fcntl\(2\)*](#)) held on the file it was associated with, and owned by the process, are removed (regardless of the file descriptor that was used to obtain the lock).

If *fd* is the last file descriptor referring to the underlying open file description (see [*open\(2\)*](#)), the resources associated with the open file description are freed; if the descriptor was the last reference to a file which has been removed using [*unlink\(2\)*](#) the file is deleted.

Return Value

close() returns zero on success. On error, -1 is returned, and *errno* is set appropriately.

Errors

EBADF

fd isn't a valid open file descriptor.

EINTR

The **close()** call was interrupted by a signal; see [*signal\(7\)*](#).

EIO

An I/O error occurred.

Conforming to

SVr4, 4.3BSD, POSIX.1-2001.

Notes

Not checking the return value of **close()** is a common but nevertheless serious programming error. It is quite possible that errors on a previous [*write\(2\)*](#) operation are first reported at the final **close()**. Not checking the return value when closing the file may lead to silent loss of data. This can especially be observed with NFS and with disk quota.



A successful close does not guarantee that the data has been successfully saved to disk, as the kernel defers writes. It is not common for a file system to flush the buffers when the stream is closed. If you need to be sure that the data is physically stored use **fsync(2)**. (It will depend on the disk hardware at this point.)

It is probably unwise to close file descriptors while they may be in use by system calls in other threads in the same process. Since a file descriptor may be reused, there are some obscure race conditions that may cause unintended side effects.

See Also

fcntl(2), **fsync(2)**, **open(2)**, **shutdown(2)**, **unlink(2)**, **fclose(3)**

Referenced By

__pmconnectlogger(3), **archive_read(3)**, **archive_write(3)**, **closedir(3)**, **cpuset(7)**, **dbopen(3)**, **dup2(2)**, **epoll(7)**, **epoll_create(2)**, **eventfd(2)**, **explain(1)**, **explain(3)**, **explain_close(3)**, **explain_close_or_die(3)**, **explain_lca2010(1)**, **fcloseall(3)**, **filter_create_fd(3)**, **flock(2)**, **fts(3)**, **getdtablesize(2)**, **getdtablesize(3)**, **inotify(7)**, **lam_rfposix(2)**, **lamf_rfclose(2)**, **mkfifo(3)**, **ncl_cgm(3)**, **netclose(3)**, **nfs(5)**, **perfmonctl(2)**, **pidfile(3)**, **pipe(7)**, **pipethrough(3)**, **read(2)**, **shm_open(3)**, **shm_overview(7)**, **signalfd(2)**, **socket(2)**, **socket(7)**, **spu_create(2)**, **spu_run(2)**, **spufs(2)**, **spufs(7)**, **star(1)**, **stdio(3)**, **tar_open(3)**, **timerfd_create(2)**



dup(2) - Linux man page



Name

dup, dup2, dup3 - duplicate a file descriptor

Synopsis

```
#include <unistd.h>
int dup(int oldfd);int dup2(int oldfd, int newfd);
#define _GNU_SOURCE          /* See feature\_test\_macros\(7\) */#include
<fcntl.h>                  /* Obtain O_* constant definitions */#include <unistd.h>
int dup3(int oldfd, int newfd, int flags);
```

Description

These system calls create a copy of the file descriptor *oldfd*.

dup() uses the lowest-numbered unused descriptor for the new descriptor.

dup2() makes *newfd* be the copy of *oldfd*, closing *newfd* first if necessary, but note the following:

- *
If *oldfd* is not a valid file descriptor, then the call fails, and *newfd* is not closed.
- *
If *oldfd* is a valid file descriptor, and *newfd* has the same value as *oldfd*, then **dup2()** does nothing, and returns *newfd*.

After a successful return from one of these system calls, the old and new file descriptors may be used interchangeably. They refer to the same open file description (see [open\(2\)](#)) and thus share file offset and file status flags; for example, if the file offset is modified by using [lseek\(2\)](#) on one of the descriptors, the offset is also changed for the other.

The two descriptors do not share file descriptor flags (the close-on-exec flag). The close-on-exec flag (**FD_CLOEXEC**; see [fcntl\(2\)](#)) for the duplicate descriptor is off.

dup3() is the same as **dup2()**, except that:

- *
The caller can force the close-on-exec flag to be set for the new file descriptor by specifying **O_CLOEXEC** in *flags*. See the description of the same flag in [open\(2\)](#) for reasons why this may be useful.
- *
If *oldfd* equals *newfd*, then **dup3()** fails with the error **EINVAL**.



Return Value

On success, these system calls return the new descriptor. On error, -1 is returned, and *errno* is set appropriately.

Errors

EBADF

oldfd isn't an open file descriptor, or *newfd* is out of the allowed range for file descriptors.

EBUSY

(Linux only) This may be returned by **dup2()** or **dup3()** during a race condition with ***open*(2)** and **dup()**.

EINTR

The **dup2()** or **dup3()** call was interrupted by a signal; see ***signal*(7)**.

EINVAL

(**dup3()**) *flags* contain an invalid value. Or, *oldfd* was equal to *newfd*.

EMFILE

The process already has the maximum number of file descriptors open and tried to open a new one.

Versions

dup3() was added to Linux in version 2.6.27; glibc support is available starting with version 2.9.

Conforming to

dup(), **dup2()**: SVr4, 4.3BSD, POSIX.1-2001.

dup3() is Linux-specific.

Notes

The error returned by **dup2()** is different from that returned by **fcntl(..., F_DUPFD, ...)** when *newfd* is out of range. On some systems **dup2()** also sometimes returns **EINVAL** like **F_DUPFD**.

If *newfd* was open, any errors that would have been reported at ***close*(2)** time are lost. A careful programmer will not use **dup2()** or **dup3()** without closing *newfd* first.

See Also

***close*(2)**, ***fcntl*(2)**, ***open*(2)**

Referenced By

epoll(7), **explain**(1), **explain**(3), **explain_dup**(3), **explain_dup_or_die**(3), **flock**(
getdtablesize(2), **getdtablesize**(3), **getrlimit**(2), **io_passfd**(3), **ksh**(1), **ksh93**(1),
lam_rfp(2), **mksh**(1), **perlfunc**(1), **perlopentut**(1), **pipe**(7), **unix**(7), **xs**(1)



execve(2) - Linux man page



Name

execve - execute program

Synopsis

#include <**unistd.h**>

```
int execve(const char *filename, char *const argv[],
char *const envp[]);
```

Description

execve() executes the program pointed to by *filename*. *filename* must be either a binary executable, or a script starting with a line of the form:

```
#! interpreter [optional-arg]
```

For details of the latter case, see "Interpreter scripts" below.

argv is an array of argument strings passed to the new program. By convention, the first of these strings should contain the filename associated with the file being executed. *envp* is an array of strings, conventionally of the form **key=value**, which are passed as environment to the new program. Both *argv* and *envp* must be terminated by a NULL pointer. The argument vector and environment can be accessed by the called program's main function, when it is defined as:

```
int main(int argc, char *argv[], char *envp[])
```

execve() does not return on success, and the text, data, bss, and stack of the calling process are overwritten by that of the program loaded.

If the current program is being ptraced, a **SIGTRAP** is sent to it after a successful **execve()**.

If the set-user-ID bit is set on the program file pointed to by *filename*, and the underlying file system is not mounted *nosuid* (the **MS_NOSUID** flag for **mount(2)**), and the calling process is not being ptraced, then the effective user ID of the calling process is changed to that of the owner of the program file. Similarly, when the set-group-ID bit of the program file is set the effective group ID of the calling process is set to the group of the program file.

The effective user ID of the process is copied to the saved set-user-ID; similarly, the effective group ID is copied to the saved set-group-ID. This copying takes place after any

effective ID changes that occur because of the set-user-ID and set-group-ID permission bits.



If the executable is an a.out dynamically linked binary executable containing shared-library stubs, the Linux dynamic linker **ld.so**(8) is called at the start of execution to bring needed shared libraries into memory and link the executable with them.

If the executable is a dynamically linked ELF executable, the interpreter named in the PT_INTERP segment is used to load the needed shared libraries. This interpreter is typically */lib/ld-linux.so.2* for binaries linked with glibc 2. (For binaries linked with the old Linux libc5, the interpreter was typically */lib/ld-linux.so.1*.)

All process attributes are preserved during an **execve()**, except the following:

*

The dispositions of any signals that are being caught are reset to the default (**signal**(7)).

*

Any alternate signal stack is not preserved (**sigaltstack**(2)).

*

Memory mappings are not preserved (**mmap**(2)).

*

Attached System V shared memory segments are detached (**shmat**(2)).

*

POSIX shared memory regions are unmapped (**shm_open**(3)).

*

Open POSIX message queue descriptors are closed (**mq_overview**(7)).

*

Any open POSIX named semaphores are closed (**sem_overview**(7)).

*

POSIX timers are not preserved (**timer_create**(2)).

*

Any open directory streams are closed (**opendir**(3)).



*

Memory locks are not preserved (**mlock**(2), **mlockall**(2)).

*

Exit handlers are not preserved (**atexit**(3), **on_exit**(3)).

*

The floating-point environment is reset to the default (see **fenv**(3)).

The process attributes in the preceding list are all specified in POSIX.1-2001. The following Linux-specific process attributes are also not preserved during an **execve**():

*

The **prctl**(2) **PR_SET_DUMPABLE** flag is set, unless a set-user-ID or set-group ID program is being executed, in which case it is cleared.

*

The **prctl**(2) **PR_SET_KEEPCAPS** flag is cleared.

*

(Since Linux 2.4.36 / 2.6.23) If a set-user-ID or set-group-ID program is being executed, then the parent death signal set by **prctl**(2) **PR_SET_PDEATHSIG** flag is cleared.

*

The process name, as set by **prctl**(2) **PR_SET_NAME** (and displayed by *ps -o comm*), is reset to the name of the new executable file.

*

The **SECBIT_KEEP_CAPS** *securebits* flag is cleared. See **capabilities**(7).

*

The termination signal is reset to **SIGCHLD** (see **clone**(2)).

Note the following further points:

*

All threads other than the calling thread are destroyed during an **execve**(). Mutexes, condition variables, and other pthreads objects are not preserved.

*

The equivalent of *setlocale(LC_ALL, "C")* is executed at program start-up.



*

POSIX.1-2001 specifies that the dispositions of any signals that are ignored or set to the default are left unchanged. POSIX.1-2001 specifies one exception: if **SIGCHLD** is being ignored, then an implementation may leave the disposition unchanged or reset it to the default; Linux does the former.

*

Any outstanding asynchronous I/O operations are canceled ([**aio_read\(3\)**](#), [**aio_write\(3\)**](#)).

*

For the handling of capabilities during **execve()**, see [**capabilities\(7\)**](#).

*

By default, file descriptors remain open across an **execve()**. File descriptors that are marked close-on-exec are closed; see the description of **FD_CLOEXEC** in [**fcntl\(2\)**](#). (If a file descriptor is closed, this will cause the release of all record locks obtained on the underlying file by this process. See [**fcntl\(2\)**](#) for details.) POSIX.1-2001 says that if file descriptors 0, 1, and 2 would otherwise be closed after a successful **execve()**, and the process would gain privilege because the set-user_ID or set-group_ID permission bit was set on the executed file, then the system may open an unspecified file for each of these file descriptors. As a general principle, no portable program, whether privileged or not, can assume that these three file descriptors will remain closed across an **execve()**.

Interpreter scripts

An interpreter script is a text file that has execute permission enabled and whose first line is of the form:

```
#! interpreter [optional-arg]
```

The *interpreter* must be a valid pathname for an executable which is not itself a script. If the *filename* argument of **execve()** specifies an interpreter script, then *interpreter* will be invoked with the following arguments:

```
interpreter [optional-arg] filename arg...
```

where *arg...* is the series of words pointed to by the *argv* argument of **execve()**.

For portable use, *optional-arg* should either be absent, or be specified as a single word (i.e., it should not contain white space); see NOTES below.

Limits on size of arguments and environment



Most UNIX implementations impose some limit on the total size of the command-line argument (*argv*) and environment (*envp*) strings that may be passed to a new program. POSIX.1 allows an implementation to advertise this limit using the **ARG_MAX** constant (either defined in [*<limits.h>*](#) or available at run time using the call `sysconf(_SC_ARG_MAX)`).

On Linux prior to kernel 2.6.23, the memory used to store the environment and argument strings was limited to 32 pages (defined by the kernel constant **MAX_ARG_PAGES**). On architectures with a 4-kB page size, this yields a maximum size of 128 kB.

On kernel 2.6.23 and later, most architectures support a size limit derived from the soft **RLIMIT_STACK** resource limit (see [`getrlimit\(2\)`](#)) that is in force at the time of the `execve()` call. (Architectures with no memory management unit are excepted: they maintain the limit that was in effect before kernel 2.6.23.) This change allows programs to have a much larger argument and/or environment list. For these architectures, the total size is limited to 1/4 of the allowed stack size. (Imposing the 1/4-limit ensures that the new program always has some stack space.) Since Linux 2.6.25, the kernel places a floor of 32 pages on this size limit, so that, even when **RLIMIT_STACK** is set very low, applications are guaranteed to have at least as much argument and environment space as was provided by Linux 2.6.23 and earlier. (This guarantee was not provided in Linux 2.6.23 and 2.6.24.) Additionally, the limit per string is 32 pages (the kernel constant **MAX_ARG_STRLEN**), and the maximum number of strings is 0x7FFFFFFF.

Return Value

On success, `execve()` does not return, on error -1 is returned, and *errno* is set appropriately.

Errors

E2BIG

The total number of bytes in the environment (*envp*) and argument list (*argv*) is too large.

EACCES

Search permission is denied on a component of the path prefix of *filename* or the name of a script interpreter. (See also [`path\_resolution\(7\)`](#).)

EACCES

The file or a script interpreter is not a regular file.

EACCES

Execute permission is denied for the file or a script or ELF interpreter.

**EACCES**

The file system is mounted *noexec*.

EFAULT

filename points outside your accessible address space.

EINVAL

An ELF executable had more than one PT_INTERP segment (i.e., tried to name more than one interpreter).

EIO

An I/O error occurred.

EISDIR

An ELF interpreter was a directory.

ELIBBAD

An ELF interpreter was not in a recognized format.

ELOOP

Too many symbolic links were encountered in resolving *filename* or the name of a script or ELF interpreter.

EMFILE

The process has the maximum number of files open.

ENAMETOOLONG

filename is too long.

ENFILE

The system limit on the total number of open files has been reached.

ENOENT

The file *filename* or a script or ELF interpreter does not exist, or a shared library needed for file or interpreter cannot be found.

ENOEXEC

An executable is not in a recognized format, is for the wrong architecture, or has some other format error that means it cannot be executed.

ENOMEM

Insufficient kernel memory was available.



ENOTDIR

A component of the path prefix of *filename* or a script or ELF interpreter is not a directory.

EPERM

The file system is mounted *nosuid*, the user is not the superuser, and the file has the set-user-ID or set-group-ID bit set.

EPERM

The process is being traced, the user is not the superuser and the file has the set-user-ID or set-group-ID bit set.

ETXTBSY

Executable was open for writing by one or more processes.

Conforming To

SVr4, 4.3BSD, POSIX.1-2001. POSIX.1-2001 does not document the `#!` behavior but is otherwise compatible.

Notes

Set-user-ID and set-group-ID processes can not be `ptrace\(2\)`.

Linux ignores the set-user-ID and set-group-ID bits on scripts.

The result of mounting a file system *nosuid* varies across Linux kernel versions: some will refuse execution of set-user-ID and set-group-ID executables when this would give the user powers she did not have already (and return **EPERM**), some will just ignore the set-user-ID and set-group-ID bits and `exec()` successfully.

A maximum line length of 127 characters is allowed for the first line in a `#!` executable shell script.

The semantics of the *optional-arg* argument of an interpreter script vary across implementations. On Linux, the entire string following the *interpreter* name is passed as a single argument to the interpreter, and this string can include white space. However, behavior differs on some other systems. Some systems use the first white space to terminate *optional-arg*. On some systems, an interpreter script can have multiple arguments, and white spaces in *optional-arg* are used to delimit the arguments.

On Linux, *argv* can be specified as NULL, which has the same effect as specifying this argument as a pointer to a list containing a single NULL pointer. **Do not take advantage of this misfeature!** It is nonstandard and nonportable: on most other UNIX systems doing this will result in an error (**EFAULT**).



POSIX.1-2001 says that values returned by **sysconf(3)** should be invariant over the lifetime of a process. However, since Linux 2.6.23, if the **RLIMIT_STACK** resource limit changes, then the value reported by **_SC_ARG_MAX** will also change, to reflect the fact that the limit on space for holding command-line arguments and environment variables has changed.

Historical

With UNIX V6 the argument list of an **exec()** call was ended by 0, while the argument list of *main* was ended by -1. Thus, this argument list was not directly usable in a further **exec()** call. Since UNIX V7 both are NULL.

Example

The following program is designed to be executed by the second program below. It just echoes its command-line one per line.

```
/* myecho.c */

#include <stdio.h>
#include <stdlib.h>

int
main(int argc, char *argv[])
{
    int j;

    for (j = 0; j < argc; j++)
        printf("argv[%d]: %s\n", j, argv[j]);

    exit(EXIT_SUCCESS);
}
```

This program can be used to exec the program named in its command-line argument:

```
/* execve.c */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int
main(int argc, char *argv[])
{
    char *newargv[] = { NULL, "hello", "world", NULL };
}
```



```
char *newenviron[] = { NULL };

if (argc != 2) {
    fprintf(stderr, "Usage: %s <file-to-exec>\n", argv[0]);
    exit(EXIT_FAILURE);
}

newargv[0] = argv[1];

execve(argv[1], newargv, newenviron);
perror("execve"); /* execve() only returns on error */
exit(EXIT_FAILURE);
}
```

We can use the second program to exec the first as follows:

```
$ cc myecho.c -o myecho
$ cc execve.c -o execve
$ ./execve ./myecho
argv[0]: ./myecho
argv[1]: hello
argv[2]: world
```

We can also use these programs to demonstrate the use of a script interpreter. To do this we create a script whose "interpreter" is our *myecho* program:

```
$ cat > script.sh
#! ./myecho script-arg
^D
$ chmod +x script.sh
```

We can then use our program to exec the script:

```
$ ./execve ./script.sh
argv[0]: ./myecho
argv[1]: script-arg
argv[2]: ./script.sh
argv[3]: hello
argv[4]: world
```

See Also

[**chmod**\(2\)](#), [**fork**\(2\)](#), [**ptrace**\(2\)](#), [**execl**\(3\)](#), [**fexecve**\(3\)](#), [**getopt**\(3\)](#), [**credentials**\(7\)](#), [**environ**\(7\)](#), [**path_resolution**\(7\)](#), [**ld.so**\(8\)](#)

Referenced By



[access\(2\)](#), [ash\(1\)](#), [cap_get_file\(3\)](#), [capset\(2\)](#), [catopen\(3\)](#), [chdir\(2\)](#), [chroot\(2\)](#), [console_ioctl\(4\)](#), [core\(5\)](#), [csh\(1\)](#), [dash\(1\)](#), [elf\(1\)](#), [elf\(5\)](#), [environ\(5\)](#), [eventfd\(2\)](#), [exec\(3\)](#), [exit\(2\)](#), [exit\(3\)](#), [expect\(1\)](#), [explain\(1\)](#), [explain\(3\)](#), [explain_execve\(3\)](#), [explain_execve_or_die\(3\)](#), [flock\(2\)](#), [getexecon\(3\)](#), [getitimer\(2\)](#), [getrusage\(2\)](#), [ioctl\(2\)](#), [ioperm\(2\)](#), [iopl\(2\)](#), [iv_signal\(3\)](#), [ksh\(1\)](#), [libexpect\(3\)](#), [lynx\(1\)](#), [mksh\(1\)](#), [mq_close\(3\)](#), [open\(2\)](#), [pmcd\(1\)](#), [proc\(5\)](#), [pthread_kill_other_threads_np\(3\)](#), [pthreads\(7\)](#), [sbrk\(2\)](#), [sched_getaffinity\(2\)](#), [sched_setscheduler\(2\)](#), [sem_close\(3\)](#), [semop\(2\)](#), [set_mempolicy\(2\)](#), [setgroups\(2\)](#), [setpgrp\(2\)](#), [setpriority\(2\)](#), [setsid\(2\)](#), [sigaction\(2\)](#), [signalfd\(2\)](#), [sigpending\(2\)](#), [sigprocmask\(2\)](#), [sigvec\(3\)](#), [stat\(2\)](#), [sudo_plugin\(8\)](#), [system\(3\)](#), [timerfd_create\(2\)](#), [umask\(2\)](#), [vfork\(2\)](#), [xs\(1\)](#)

exit(2) - Linux man page



Name

`_exit`, `_Exit` - terminate the calling process

Synopsis

#include <[unistd.h](#)>

void `_exit`(int *status*);

#include <[stdlib.h](#)>

void `_Exit`(int *status*);

Feature Test Macro Requirements for glibc (see [feature_test_macros\(7\)](#)):

`_Exit`():

`_XOPEN_SOURCE` >= 600 || `_ISOC99_SOURCE` || `_POSIX_C_SOURCE` >= 200112L;
or `cc -std=c99`

Description

The function `_exit`() terminates the calling process "immediately". Any open file descriptors belonging to the process are closed; any children of the process are inherited by process 1, *init*, and the process's parent is sent a **SIGCHLD** signal.

The value *status* is returned to the parent process as the process's exit status, and can be collected using one of the [wait](#)(2) family of calls.

The function `_Exit`() is equivalent to `_exit`().

Return Value

These functions do not return.

Conforming To

SVr4, POSIX.1-2001, 4.3BSD. The function `_Exit`() was introduced by C99.

Notes

For a discussion on the effects of an exit, the transmission of exit status, zombie processes, signals sent, and so on, see [exit](#)(3).

The function `_exit`() is like [exit](#)(3), but does not call any functions registered with [atexit](#)(3) or [on_exit](#)(3). Whether it flushes standard I/O buffers and removes temporary

files created with [tmpfile\(3\)](#) is implementation-dependent. On the other hand, [_exit\(2\)](#) does close open file descriptors, and this may cause an unknown delay, waiting for pending output to finish. If the delay is undesired, it may be useful to call functions like [tcflush\(3\)](#) before calling [_exit\(\)](#). Whether any pending I/O is canceled, and which pending I/O may be canceled upon [_exit\(\)](#), is implementation-dependent.



In glibc up to version 2.3, the [_exit\(\)](#) wrapper function invoked the kernel system call of the same name. Since glibc 2.3, the wrapper function invokes [exit_group\(2\)](#), in order to terminate all of the threads in a process.

See Also

[execve\(2\)](#), [exit_group\(2\)](#), [fork\(2\)](#), [kill\(2\)](#), [wait\(2\)](#), [wait4\(2\)](#), [waitpid\(2\)](#), [atexit\(3\)](#), [exit\(3\)](#), [on_exit\(3\)](#), [termios\(3\)](#)

Referenced By

[allegro_message\(3\)](#), [clone\(2\)](#), [drmaa_wait\(3\)](#), [explain_accept4_or_die\(3\)](#), [explain_accept_or_die\(3\)](#), [explain_acct_or_die\(3\)](#), [explain_adjtime_or_die\(3\)](#), [explain_adjtimex_or_die\(3\)](#), [explain_bind_or_die\(3\)](#), [explain_calloc_or_die\(3\)](#), [explain_chown_or_die\(3\)](#), [explain_chroot_or_die\(3\)](#), [explain_close_or_die\(3\)](#), [explain_closedir_or_die\(3\)](#), [explain_connect_or_die\(3\)](#), [explain_creat_or_die\(3\)](#), [explain_dirfd_or_die\(3\)](#), [explain_dup2_or_die\(3\)](#), [explain_dup_or_die\(3\)](#), [explain_eventfd_or_die\(3\)](#), [explain_execlp_or_die\(3\)](#), [explain_execve_or_die\(3\)](#), [explain_execlp_or_die\(3\)](#), [explain_fchdir_or_die\(3\)](#), [explain_fchown_or_die\(3\)](#), [explain_fcntl_or_die\(3\)](#), [explain_fdopen_or_die\(3\)](#), [explain_fdopendir_or_die\(3\)](#), [explain_feof_or_die\(3\)](#), [explain_ferror_or_die\(3\)](#), [explain_fflush_or_die\(3\)](#), [explain_fgetc_or_die\(3\)](#), [explain_fgetpos_or_die\(3\)](#), [explain_fgets_or_die\(3\)](#), [explain_filenos_or_die\(3\)](#), [explain_flock_or_die\(3\)](#), [explain_fopen_or_die\(3\)](#), [explain_fork_or_die\(3\)](#), [explain_fpathconf_or_die\(3\)](#), [explain_fprintf_or_die\(3\)](#), [explain_fpurge_or_die\(3\)](#), [explain_fputc_or_die\(3\)](#), [explain_fputs_or_die\(3\)](#), [explain_fread_or_die\(3\)](#), [explain_fseek_or_die\(3\)](#), [explain_fsetpos_or_die\(3\)](#), [explain_fstat_or_die\(3\)](#), [explain_fstatfs_or_die\(3\)](#), [explain_fstatvfs_or_die\(3\)](#), [explain_fsync_or_die\(3\)](#), [explain_ftell_or_die\(3\)](#), [explain_ftime_or_die\(3\)](#), [explain_ftruncate_or_die\(3\)](#), [explain_futimes_or_die\(3\)](#), [explain_fwrote_or_die\(3\)](#), [explain_getaddrinfo_or_die\(3\)](#), [explain_getc_or_die\(3\)](#), [explain_getchar_or_die\(3\)](#), [explain_getcwd_or_die\(3\)](#), [explain_getdomainname_or_die\(3\)](#), [explain_getgroups_or_die\(3\)](#), [explain_gethostname_or_die\(3\)](#), [explain_getpeername_or_die\(3\)](#), [explain_getpgid_or_die\(3\)](#), [explain_getpgrp_or_die\(3\)](#), [explain_getrlimit_or_die\(3\)](#), [explain_getsockname_or_die\(3\)](#), [explain_getsockopt_or_die\(3\)](#), [explain_gettimeofday_or_die\(3\)](#), [explain_getw_or_die\(3\)](#), [explain_ioctl_or_die\(3\)](#), [explain_kill_or_die\(3\)](#), [explain_lca2010\(1\)](#), [explain_lchown_or_die\(3\)](#), [explain_link_or_die\(3\)](#),



explain listen or die(3), explain lseek or die(3), explain lstat or die(3),
explain malloc or die(3), explain mkdir or die(3), explain mkdtemp or die(3),
explain mknod or die(3), explain mkostemp or die(3),
explain mkstemp or die(3), explain mktemp or die(3),
explain mmap or die(3), explain munmap or die(3), explain nice or die(3),
explain opendir or die(3), explain output(3), explain pathconf or die(3),
explain pclose or die(3), explain pipe or die(3), explain poll or die(3),
explain popen or die(3), explain pread or die(3), explain printf or die(3),
explain ptrace or die(3), explain putc or die(3), explain putchar or die(3),
explain putenv or die(3), explain puts or die(3), explain putw or die(3),
explain pwrite or die(3), explain raise or die(3), explain read or die(3),
explain readdir or die(3), explain readlink or die(3), explain readv or die(3),
explain realloc or die(3), explain realpath or die(3),
explain remove or die(3), explain rename or die(3), explain rmdir or die(3),
explain select or die(3), explain setbuf or die(3), explain setbuffer or die(3),
explain setdomainname or die(3), explain setenv or die(3),
explain setgroups or die(3), explain sethostname or die(3),
explain setlinebuf or die(3), explain setpgid or die(3),
explain setpgrp or die(3), explain setsid or die(3),
explain setsockopt or die(3), explain setvbuf or die(3),
explain shmat or die(3), explain shmctl or die(3), explain signalfd or die(3),
explain snprintf or die(3), explain socket or die(3),
explain socketpair or die(3), explain sprintf or die(3), explain stat or die(3),
explain statfs or die(3), explain statvfs or die(3), explain stime or die(3),
explain strdup or die(3), explain strndup or die(3), explain strtod or die(3),
explain strtod or die(3), explain strtol or die(3), explain strtold or die(3),
explain strtoll or die(3), explain strtoul or die(3), explain strtoull or die(3),
explain symlink or die(3), explain system or die(3), explain tcdrain or die(3),
explain tcflow or die(3), explain tcflush or die(3), explain tcgetattr or die(3),
explain tcsendbreak or die(3), explain tcsetattr or die(3),
explain telldir or die(3), explain tempnam or die(3), explain time or die(3),
explain timerfd_create or die(3), explain tmpfile or die(3),
explain tmpnam or die(3), explain truncate or die(3),
explain ungetc or die(3), explain unlink or die(3), explain unsetenv or die(3),
explain ustat or die(3), explain utime or die(3), explain utimes or die(3),
explain vfork or die(3), explain vfprintf or die(3), explain vprintf or die(3),
explain vsnprintf or die(3), explain vsprintf or die(3),
explain wait3 or die(3), explain wait4 or die(3), explain wait or die(3),
explain waitpid or die(3), explain write or die(3), explain writev or die(3),
pagesend(8), pmgetconfig(3), ptrace(2), socket(7)

fork(2) - Linux man page

Name

fork - create a child process

Synopsis

#include <unistd.h>

pid_t fork(void);

Description

fork() creates a new process by duplicating the calling process. The new process, referred to as the *child*, is an exact duplicate of the calling process, referred to as the *parent*, except for the following points:

- *
The child has its own unique process ID, and this PID does not match the ID of any existing process group (setpgid(2)).
- *
The child's parent process ID is the same as the parent's process ID.
- *
The child does not inherit its parent's memory locks (mlock(2), mlockall(2)).
- *
Process resource utilizations (getrusage(2)) and CPU time counters (times(2)) are reset to zero in the child.
- *
The child's set of pending signals is initially empty (sigpending(2)).
- *
The child does not inherit semaphore adjustments from its parent (semop(2)).
- *
The child does not inherit record locks from its parent (fcntl(2)).
- *
The child does not inherit timers from its parent (setitimer(2), alarm(2), timer_create(2)).
- *
The child does not inherit outstanding asynchronous I/O operations from its parent (aio_read(3), aio_write(3)), nor does it inherit any asynchronous I/O contexts from its parent (see io_setup(2)).

The process attributes in the preceding list are all specified in POSIX.1-2001. The parent and child also differ with respect to the following Linux-specific process attributes:



*

The child does not inherit directory change notifications (dnotify) from its parent (see the description of **F_NOTIFY** in [*fcntl\(2\)*](#)).

*

The [*prctl\(2\)*](#) **PR_SET_PDEATHSIG** setting is reset so that the child does not receive a signal when its parent terminates.

*

The default timer slack value is set to the parent's current timer slack value. See the description of **PR_SET_TIMERSLACK** in [*prctl\(2\)*](#).

*

Memory mappings that have been marked with the [*madvise\(2\)*](#) **MADV_DONTFORK** flag are not inherited across a **fork()**.

*

The termination signal of the child is always **SIGCHLD** (see [*clone\(2\)*](#)).

*

The port access permission bits set by [*ioperm\(2\)*](#) are not inherited by the child; the child must turn on any bits that it requires using [*ioperm\(2\)*](#).

Note the following further points:

*

The child process is created with a single thread--the one that called **fork()**. The entire virtual address space of the parent is replicated in the child, including the : of mutexes, condition variables, and other pthreads objects; the use of [*pthread_atfork\(3\)*](#) may be helpful for dealing with problems that this can cause.



*

The child inherits copies of the parent's set of open file descriptors. Each file descriptor in the child refers to the same open file description (see [*open\(2\)*](#)) as the corresponding file descriptor in the parent. This means that the two descriptors share open file status flags, current file offset, and signal-driven I/O attributes (see the description of **F_SETOWN** and **F_SETSIG** in [*fcntl\(2\)*](#)).

*

The child inherits copies of the parent's set of open message queue descriptors (see [*mq_overview\(7\)*](#)). Each descriptor in the child refers to the same open message queue description as the corresponding descriptor in the parent. This means that the two descriptors share the same flags (*mq_flags*).

*

The child inherits copies of the parent's set of open directory streams (see [*opendir\(3\)*](#)). POSIX.1-2001 says that the corresponding directory streams in the parent and child *may* share the directory stream positioning; on Linux/glibc they do not.

Return Value

On success, the PID of the child process is returned in the parent, and 0 is returned in the child. On failure, -1 is returned in the parent, no child process is created, and *errno* is set appropriately.

Errors

EAGAIN

fork() cannot allocate sufficient memory to copy the parent's page tables and allocate a task structure for the child.

EAGAIN

It was not possible to create a new process because the caller's **RLIMIT_NPROC** resource limit was encountered. To exceed this limit, the process must have either the **CAP_SYS_ADMIN** or the **CAP_SYS_RESOURCE** capability.

ENOMEM

fork() failed to allocate the necessary kernel structures because memory is tight.

ENOSYS

fork() is not supported on this platform (for example, hardware without a Memory-Management Unit).

Conforming to

SVr4, 4.3BSD, POSIX.1-2001.

Notes



Under Linux, **fork()** is implemented using copy-on-write pages, so the only penalty that it incurs is the time and memory required to duplicate the parent's page tables, and to create a unique task structure for the child.

Since version 2.3.3, rather than invoking the kernel's **fork()** system call, the glibc **fork()** wrapper that is provided as part of the NPTL threading implementation invokes **clone(2)** with flags that provide the same effect as the traditional system call. (A call to **fork()** is equivalent to a call to **clone(2)** specifying *flags* as just **SIGCHLD**.) The glibc wrapper invokes any fork handlers that have been established using **pthread_atfork(3)**.

Example

See **pipe(2)** and **wait(2)**.

See Also

clone(2), **execve(2)**, **exit(2)**, **setrlimit(2)**, **unshare(2)**, **vfork(2)**, **wait(2)**, **daemon(3)**, **capabilities(7)**, **credentials(7)**

Referenced By

atexit(3), **chdir**(2), **chroot**(2), **collectd-exec**(5), **core**(5), **cpuset**(7), **csch**(1), **dbpmda**(1), **dc_client**(1), **dc_ctx_new**(2), **dkim-filter**(8), **dkim-filter.conf**(5), **epic**(1), **epoll**(7), **eventfd**(2), **exec**(3), **explain**(1), **explain**(3), **explain_fork**(3), **explain_fork_or_die**(3), **faxq**(8), **fetchnews**(8), **fio**(1), **flock**(2), **forkpty**(3), **getitimer**(2), **getpid**(2), **getrlimit**(2), **gettid**(2), **halockrun**(1), **hylafax-log**(5), **ibv_fork_init**(3), **inn.conf**(5), **iopl**(2), **iv_signal**(3), **ksh93**(1), **lseek**(2), **ltrace**(1), **mmap**(2), **mount**(2), **mpadvise**(3), **nice**(2), **on_exit**(3), **opendkim**(8), **opendkim.conf**(5), **pam_end**(3), **perlfunc**(1), **pipe**(7), **pmcd**(1), **popen**(3), **proc**(5), **pth**(3), **pthreads**(7), **ptrace**(2), **sched_getaffinity**(2), **sched_setscheduler**(2), **sec**(1), **select_tut**(2), **sem_init**(3), **set_mempolicy**(2), **setns**(2), **setpggrp**(2), **setpriority**(2), **setsid**(2), **shmat**(2), **sigaction**(2), **sigaltstack**(2), **signal**(7), **signalfd**(2), **sigprocmask**(2), **smartd**(8), **smartd.conf**(5), **strace**(1), **sudo**(8), **syslogd**(8), **system**(3), **timerfd_create**(2), **umask**(2), **wait4**(2), **xargs**(1)



lseek(2) - Linux man page

Name

lseek - reposition read/write file offset

Synopsis

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
off_t lseek(int fd, off_t offset, int whence);
```

Description

The **lseek()** function repositions the offset of the open file associated with the file descriptor *fd* to the argument *offset* according to the directive *whence* as follows:

SEEK_SET

The offset is set to *offset* bytes.

SEEK_CUR

The offset is set to its current location plus *offset* bytes.

SEEK_END

The offset is set to the size of the file plus *offset* bytes.



The **lseek()** function allows the file offset to be set beyond the end of the file (but this does not change the size of the file). If data is later written at this point, subsequent reads of the data in the gap (a "hole") return null bytes (aq\0aq) until data is actually written into the gap.

Seeking file data and holes

Since version 3.1, Linux supports the following additional values for *whence*:

SEEK_DATA

Adjust the file offset to the next location in the file greater than or equal to *offset* containing data. If *offset* points to data, then the file offset is set to *offset*.

SEEK_HOLE

Adjust the file offset to the next hole in the file greater than or equal to *offset*. If *offset* points into the middle of a hole, then the file offset is set to *offset*. If there is no hole past *offset*, then the file offset is adjusted to the end of the file (i.e., there is an implicit hole at the end of any file).

In both of the above cases, **lseek()** fails if *offset* points past the end of the file.

These operations allow applications to map holes in a sparsely allocated file. This can be useful for applications such as file backup tools, which can save space when creating backups and preserve holes, if they have a mechanism for discovering holes.

For the purposes of these operations, a hole is a sequence of zeros that (normally) has not been allocated in the underlying file storage. However, a file system is not obliged to report holes, so these operations are not a guaranteed mechanism for mapping the storage space actually allocated to a file. (Furthermore, a sequence of zeros that actually has been written to the underlying storage may not be reported as a hole.) In the simplest implementation, a file system can support the operations by making **SEEK_HOLE** always return the offset of the end of the file, and making **SEEK_DATA** always return *offset* (i.e., even if the location referred to by *offset* is a hole, it can be considered to consist of data that is a sequence of zeros).

Return Value

Upon successful completion, **lseek()** returns the resulting offset location as measured in bytes from the beginning of the file. On error, the value (*off_t*) *-1* is returned and *errno* is set to indicate the error.

Errors

EBADF

fd is not an open file descriptor.

EINVAL

whence is not valid. Or: the resulting file offset would be negative, or beyond the end of a seekable device.

EOVERFLOW

The resulting file offset cannot be represented in an *off_t*.

ESPIPE

fd is associated with a pipe, socket, or FIFO.

ENXIO

whence is **SEEK_DATA** or **SEEK_HOLE**, and the current file offset is beyond the end of the file.

Conforming to

SVr4, 4.3BSD, POSIX.1-2001.

SEEK_DATA and **SEEK_HOLE** are nonstandard extensions also present in Solaris, FreeBSD, and DragonFly BSD; they are proposed for inclusion in the next POSIX revision (Issue 8).

Notes



Some devices are incapable of seeking and POSIX does not specify which devices must support **lseek()**.

On Linux, using **lseek()** on a terminal device returns **ESPIPE**.

When converting old code, substitute values for *whence* with the following macros:

old	new
0	SEEK_SET
1	SEEK_CUR
2	SEEK_END
L_SET	SEEK_SET
L_INCR	SEEK_CUR
L_XTND	SEEK_END

Note that file descriptors created by **dup(2)** or **fork(2)** share the current file position pointer, so seeking on such files may be subject to race conditions.

See Also

dup(2), **fork(2)**, **open(2)**, **fseek(3)**, **lseek64(3)**, **posix_fallocate(3)**

Referenced By

_llseek(2), **afio(1)**, **cpuid(4)**, **dnet(3)**, **dup2(2)**, **explain(1)**, **explain(3)**, **explain_lseek(3)**, **explain_lseek_or_die(3)**, **explain_open_or_die(3)**, **fsgrab(1)**, **getdirenties(3)**, **ksh93(1)**, **lam_rfposix(2)**, **myrescue(1)**, **ncl_cgm(3)**, **open_append(3)**, **perl5004delta(1)**, **perlfunc(1)**, **pipe(7)**, **pread(2)**, **proc(5)**, **gio(3)**, **read(2)**, **readahead(2)**, **rmt(8)**, **seekdir(3)**, **spufs(2)**, **spufs(7)**, **star(1)**, **stdin(3)**, **tiffopen(3)**, **vche(1)**, **write(2)**, **writew(2)**, **x11vnc(1)**, **xs(1)**, **xxd(1)**, **zshexpn(1)**



mremap(2) - Linux man page



Name

mremap - remap a virtual memory address

Synopsis

```
#define _GNU_SOURCE          /* See feature\_test\_macros\(7\) */
#include <sys/mman.h>
void *mremap(void *old_address, size_t old_size,          size_t new_size,
int flags, ... /* void *new_address */);
```

Description

mremap() expands (or shrinks) an existing memory mapping, potentially moving it at the same time (controlled by the *flags* argument and the available virtual address space).

old_address is the old address of the virtual memory block that you want to expand (or shrink). Note that *old_address* has to be page aligned. *old_size* is the old size of the virtual memory block. *new_size* is the requested size of the virtual memory block after the resize. An optional fifth argument, *new_address*, may be provided; see the description of **MREMAP_FIXED** below.

In Linux the memory is divided into pages. A user process has (one or) several linear virtual memory segments. Each virtual memory segment has one or more mappings to real memory pages (in the page table). Each virtual memory segment has its own protection (access rights), which may cause a segmentation violation if the memory is accessed incorrectly (e.g., writing to a read-only segment). Accessing virtual memory outside of the segments will also cause a segmentation violation.

mremap() uses the Linux page table scheme. **mremap()** changes the mapping between virtual addresses and memory pages. This can be used to implement a very efficient [realloc\(3\)](#).

The *flags* bit-mask argument may be 0, or include the following flag:

MREMAP_MAYMOVE

By default, if there is not sufficient space to expand a mapping at its current location, then **mremap()** fails. If this flag is specified, then the kernel is permitted to relocate the mapping to a new virtual address, if necessary. If the mapping is relocated, then absolute pointers into the old mapping location become invalid (offsets relative to the starting address of the mapping should be employed).

MREMAP_FIXED (since Linux 2.3.31)



This flag serves a similar purpose to the **MAP_FIXED** flag of **mmap(2)**. If this flag is specified, then **mremap()** accepts a fifth argument, *void *new_address*, which specifies a page-aligned address to which the mapping must be moved. Any previous mapping at the address range specified by *new_address* and *new_size* is unmapped. If **MREMAP_FIXED** is specified, then **MREMAP_MAYMOVE** must also be specified.

If the memory segment specified by *old_address* and *old_size* is locked (using **mlock(2)** or similar), then this lock is maintained when the segment is resized and/or relocated. As a consequence, the amount of memory locked by the process may change.

Return Value

On success **mremap()** returns a pointer to the new virtual memory area. On error, the value **MAP_FAILED** (that is, *(void *) -1*) is returned, and *errno* is set appropriately.

Errors

EAGAIN

The caller tried to expand a memory segment that is locked, but this was not possible without exceeding the **RLIMIT_MEMLOCK** resource limit.

EFAULT

"Segmentation fault." Some address in the range *old_address* to *old_address+old_size* is an invalid virtual memory address for this process. You can also get **EFAULT** even if there exist mappings that cover the whole address space requested, but those mappings are of different types.

EINVAL

An invalid argument was given. Possible causes are: *old_address* was not page aligned; a value other than **MREMAP_MAYMOVE** or **MREMAP_FIXED** was specified in *flags*; *new_size* was zero; *new_size* or *new_address* was invalid; or the new address range specified by *new_address* and *new_size* overlapped the old address range specified by *old_address* and *old_size*; or **MREMAP_FIXED** was specified without also specifying **MREMAP_MAYMOVE**.

ENOMEM

The memory area cannot be expanded at the current virtual address, and the **MREMAP_MAYMOVE** flag is not set in *flags*. Or, there is not enough (virtual) memory available.

Conforming to

This call is Linux-specific, and should not be used in programs intended to be portable.

Notes

Prior to version 2.4, glibc did not expose the definition of **MREMAP_FIXED**, and the prototype for **mremap()** did not allow for the *new_address* argument.

See Also



brk(2), **getpagesize**(2), **getrlimit**(2), **mlock**(2), **mmap**(2), **sbrk**(2), **malloc**(3), **realloc**(3)

Your favorite text book on operating systems for more information on paged memory (e.g., *Modern Operating Systems* by Andrew S. Tanenbaum, *Inside Linux* by Randolph Bentson, *The Design of the UNIX Operating System* by Maurice J. Bach)

Referenced By

mmap2(2), **remap_file_pages**(2)

munmap(2) - Linux man page



Name

mmap, munmap - map or unmap files or devices into memory

Synopsis

```
#include <sys/mman.h>
void *mmap(void *addr, size_t length, int " prot ", int " flags ",
           int fd, off_t offset);
int munmap(void *addr, size_t length);
```

See NOTES for information on feature test macro requirements.

Description

mmap() creates a new mapping in the virtual address space of the calling process. The starting address for the new mapping is specified in *addr*. The *length* argument specifies the length of the mapping.

If *addr* is NULL, then the kernel chooses the address at which to create the mapping; this is the most portable method of creating a new mapping. If *addr* is not NULL, then the kernel takes it as a hint about where to place the mapping; on Linux, the mapping will be created at a nearby page boundary. The address of the new mapping is returned as the result of the call.

The contents of a file mapping (as opposed to an anonymous mapping; see **MAP_ANONYMOUS** below), are initialized using *length* bytes starting at offset *offset* in the file (or other object) referred to by the file descriptor *fd*. *offset* must be a multiple of the page size as returned by *sysconf(_SC_PAGE_SIZE)*.

The *prot* argument describes the desired memory protection of the mapping (and must not conflict with the open mode of the file). It is either **PROT_NONE** or the bitwise OR of one or more of the following flags:

PROT_EXEC

Pages may be executed.

PROT_READ

Pages may be read.

PROT_WRITE

Pages may be written.

PROT_NONE

Pages may not be accessed.

The *flags* argument determines whether updates to the mapping are visible to other processes mapping the same region, and whether updates are carried through to the

underlying file. This behavior is determined by including exactly one of the following values in *flags*:



MAP_SHARED

Share this mapping. Updates to the mapping are visible to other processes that map this file, and are carried through to the underlying file. The file may not actually be updated until ***msync***(2) or ***munmap***() is called.

MAP_PRIVATE

Create a private copy-on-write mapping. Updates to the mapping are not visible to other processes mapping the same file, and are not carried through to the underlying file. It is unspecified whether changes made to the file after the ***mmap***() call are visible in the mapped region.

Both of these flags are described in POSIX.1-2001.

In addition, zero or more of the following values can be ORed in *flags*:

MAP_32BIT (since Linux 2.4.20, 2.6)

Put the mapping into the first 2 Gigabytes of the process address space. This flag is only supported on x86-64, for 64-bit programs. It was added to allow thread stacks to be allocated somewhere in the first 2GB of memory, so as to improve context-switch performance on some early 64-bit processors. Modern x86-64 processors no longer have this performance problem, so use of this flag is not required on those systems. The **MAP_32BIT** flag is ignored when **MAP_FIXED** is set.

MAP_ANON

Synonym for **MAP_ANONYMOUS**. Deprecated.

MAP_ANONYMOUS

The mapping is not backed by any file; its contents are initialized to zero. The *fd* and *offset* arguments are ignored; however, some implementations require *fd* to be -1 if **MAP_ANONYMOUS** (or **MAP_ANON**) is specified, and portable applications should ensure this. The use of **MAP_ANONYMOUS** in conjunction with **MAP_SHARED** is only supported on Linux since kernel 2.4.

MAP_DENYWRITE

This flag is ignored. (Long ago, it signaled that attempts to write to the underlying file should fail with **ETXTBUSY**. But this was a source of denial-of-service attacks.)

MAP_EXECUTABLE

This flag is ignored.

MAP_FILE

Compatibility flag. Ignored.

MAP_FIXED

Don't interpret *addr* as a hint: place the mapping at exactly that address. *addr* must be a multiple of the page size. If the memory region specified by *addr* and *len* overlaps pages of any existing **mapping**(s), then the overlapped part of the existing **mapping**(s) will be discarded. If the specified address cannot be used, **mmap**() will



fail. Because requiring a fixed address for a mapping is less portable, the use of this option is discouraged.

MAP_GROWSDOWN

Used for stacks. Indicates to the kernel virtual memory system that the mapping should extend downward in memory.

MAP_HUGETLB (since Linux 2.6.32)

Allocate the mapping using "huge pages." See the Linux kernel source file *Documentation/vm/hugetlbpage.txt* for further information.

MAP_LOCKED (since Linux 2.5.37)

Lock the pages of the mapped region into memory in the manner of **mlock(2)**. This flag is ignored in older kernels.

MAP_NONBLOCK (since Linux 2.5.46)

Only meaningful in conjunction with **MAP_POPULATE**. Don't perform read-ahead: only create page table entries for pages that are already present in RAM. Since Linux 2.6.23, this flag causes **MAP_POPULATE** to do nothing. One day the combination of **MAP_POPULATE** and **MAP_NONBLOCK** may be reimplemented.

MAP_NORESERVE

Do not reserve swap space for this mapping. When swap space is reserved, one has the guarantee that it is possible to modify the mapping. When swap space is not reserved one might get **SIGSEGV** upon a write if no physical memory is available. See also the discussion of the file */proc/sys/vm/overcommit_memory* in **proc(5)**. In kernels before 2.6, this flag only had effect for private writable mappings.

MAP_POPULATE (since Linux 2.5.46)

Populate (prefault) page tables for a mapping. For a file mapping, this causes read-ahead on the file. Later accesses to the mapping will not be blocked by page faults.

MAP_POPULATE is only supported for private mappings since Linux 2.6.23.

MAP_STACK (since Linux 2.6.27)

Allocate the mapping at an address suitable for a process or thread stack. This flag is currently a no-op, but is used in the glibc threading implementation so that if some architectures require special treatment for stack allocations, support can later be transparently implemented for glibc.

MAP_UNINITIALIZED (since Linux 2.6.33)

Don't clear anonymous pages. This flag is intended to improve performance on embedded devices. This flag is only honored if the kernel was configured with the **CONFIG_MMAP_ALLOW_UNINITIALIZED** option. Because of the security implications, that option is normally enabled only on embedded devices (i.e., devices where one has complete control of the contents of user memory).

Of the above flags, only **MAP_FIXED** is specified in POSIX.1-2001. However, most systems also support **MAP_ANONYMOUS** (or its synonym **MAP_ANON**).

Some systems document the additional flags **MAP_AUTOGROW**, **MAP_AUTORESrv**, **MAP_COPY**, and **MAP_LOCAL**.

Memory mapped by **mmap()** is preserved across **fork(2)**, with the same attributes.



A file is mapped in multiples of the page size. For a file that is not a multiple of the page size, the remaining memory is zeroed when mapped, and writes to that region are not written out to the file. The effect of changing the size of the underlying file of a mapping on the pages that correspond to added or removed regions of the file is unspecified.

munmap()

The **munmap()** system call deletes the mappings for the specified address range, and causes further references to addresses within the range to generate invalid memory references. The region is also automatically unmapped when the process is terminated. On the other hand, closing the file descriptor does not unmap the region.

The address *addr* must be a multiple of the page size. All pages containing a part of the indicated range are unmapped, and subsequent references to these pages will generate **SIGSEGV**. It is not an error if the indicated range does not contain any mapped pages.

Timestamps changes for file-backed mappings

For file-backed mappings, the *st_atime* field for the mapped file may be updated at any time between the **mmap()** and the corresponding unmapping; the first reference to a mapped page will update the field if it has not been already.

The *st_ctime* and *st_mtime* field for a file mapped with **PROT_WRITE** and **MAP_SHARED** will be updated after a write to the mapped region, and before a subsequent **msync(2)** with the **MS_SYNC** or **MS_ASYNC** flag, if one occurs.

Return Value

On success, **mmap()** returns a pointer to the mapped area. On error, the value **MAP_FAILED** (that is, *(void *) -1*) is returned, and *errno* is set appropriately. On success, **munmap()** returns 0, on failure -1, and *errno* is set (probably to **EINVAL**).

Errors

EACCES

A file descriptor refers to a non-regular file. Or **MAP_PRIVATE** was requested, but *fd* is not open for reading. Or **MAP_SHARED** was requested and **PROT_WRITE** is set, but *fd* is not open in read/write (**O_RDWR**) mode. Or **PROT_WRITE** is set, but the file is append-only.

EAGAIN

The file has been locked, or too much memory has been locked (see **setrlimit(2)**).

EBADF

fd is not a valid file descriptor (and **MAP_ANONYMOUS** was not set).

**EINVAL**

We don't like *addr*, *length*, or *offset* (e.g., they are too large, or not aligned on a boundary).

EINVAL

(since Linux 2.6.12) *length* was 0.

EINVAL

flags contained neither **MAP_PRIVATE** or **MAP_SHARED**, or contained both of these values.

ENFILE

The system limit on the total number of open files has been reached.

ENODEV

The underlying file system of the specified file does not support memory mapping.

ENOMEM

No memory is available, or the process's maximum number of mappings would have been exceeded.

EPERM

The *prot* argument asks for **PROT_EXEC** but the mapped area belongs to a file on a file system that was mounted no-exec.

ETXTBSY

MAP_DENYWRITE was set but the object specified by *fd* is open for writing.

EOVERFLOW

On 32-bit architecture together with the large file extension (i.e., using 64-bit *off_t*): the number of pages used for *length* plus number of pages used for *offset* would overflow *unsigned long* (32 bits).

Use of a mapped region can result in these signals:

SIGSEGV

Attempted write into a region mapped as read-only.

SIGBUS

Attempted access to a portion of the buffer that does not correspond to the file (for example, beyond the end of the file, including the case where another process has truncated the file).

Conforming to

SVr4, 4.4BSD, POSIX.1-2001.

Availability

On POSIX systems on which **mmap()**, ***msync*(2)** and **munmap()** are available, **_POSIX_MAPPED_FILES** is defined in [*<unistd.h>*](#) to a value greater than 0. (See also ***sysconf*(3)**.)

Notes



This page describes the interface provided by the glibc **mmap()** wrapper function. Originally, this function invoked a system call of the same name. Since kernel 2.4, the system call has been superseded by **mmap2(2)**, and nowadays the glibc **mmap()** wrapper function invokes **mmap2(2)** with a suitably adjusted value for *offset*.

On some hardware architectures (e.g., i386), **PROT_WRITE** implies **PROT_READ**. It is architecture dependent whether **PROT_READ** implies **PROT_EXEC** or not. Portable programs should always set **PROT_EXEC** if they intend to execute code in the new mapping.

The portable way to create a mapping is to specify *addr* as 0 (NULL), and omit **MAP_FIXED** from *flags*. In this case, the system chooses the address for the mapping; the address is chosen so as not to conflict with any existing mapping, and will not be 0. If the **MAP_FIXED** flag is specified, and *addr* is 0 (NULL), then the mapped address will be 0 (NULL).

Certain *flags* constants are defined only if either **_BSD_SOURCE** or **_SVID_SOURCE** is defined. (Requiring **_GNU_SOURCE** also suffices, and requiring that macro specifically would have been more logical, since these flags are all Linux specific.) The relevant flags are: **MAP_32BIT**, **MAP_ANONYMOUS** (and the synonym **MAP_ANON**), **MAP_DENYWRITE**, **MAP_EXECUTABLE**, **MAP_FILE**, **MAP_GROWSDOWN**, **MAP_HUGETLB**, **MAP_LOCKED**, **MAP_NONBLOCK**, **MAP_NORESERVE**, **MAP_POPULATE**, and **MAP_STACK**.

Bugs

On Linux there are no guarantees like those suggested above under **MAP_NORESERVE**. By default, any process can be killed at any moment when the system runs out of memory.

In kernels before 2.6.7, the **MAP_POPULATE** flag only has effect if *prot* is specified as **PROT_NONE**.

SUSv3 specifies that **mmap()** should fail if *length* is 0. However, in kernels before 2.6.12, **mmap()** succeeded in this case: no mapping was created and the call returned *addr*. Since kernel 2.6.12, **mmap()** fails with the error **EINVAL** for this case.

POSIX specifies that the system shall always zero fill any partial page at the end of the object and that system will never write any modification of the object beyond its end. On Linux, when you write data to such partial page after the end of the object, the data stays in the page cache even after the file is closed and unmapped and even though the data is never written to the file itself, subsequent mappings may see the modified content. In some cases, this could be fixed by calling **msync(2)** before the unmap takes place; however, this doesn't work on tmpfs (for example, when using POSIX shared memory interface documented in [**shm overview\(7\)**](#)).

Example

The following program prints part of the file specified in its first command-line argument to standard output. The range of bytes to be printed is specified via offset and length values in the second and third command-line arguments. The program creates a memory mapping of the required pages of the file and then uses **write**(2) to output the desired bytes.



```
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#define handle_error(msg) \
    do { perror(msg); exit(EXIT_FAILURE); } while (0)
int
main(int argc, char *argv[])
{
    char *addr;
    int fd;
    struct stat sb;
    off_t offset, pa_offset;
    size_t length;
    ssize_t s;
    if (argc < 3 || argc > 4) {
        fprintf(stderr, "%s file offset [length]\n", argv[0]);
        exit(EXIT_FAILURE);
    }
    fd = open(argv[1], O_RDONLY);
    if (fd == -1)
        handle_error("open");
    if (fstat(fd, &sb) == -1)           /* To obtain file size */
        handle_error("fstat");
    offset = atoi(argv[2]);
    pa_offset = offset & ~(sysconf(_SC_PAGE_SIZE) - 1);
    /* offset for mmap() must be page aligned */
    if (offset >= sb.st_size) {
        fprintf(stderr, "offset is past end of file\n");
        exit(EXIT_FAILURE);
    }
    if (argc == 4) {
        length = atoi(argv[3]);
        if (offset + length > sb.st_size)
            length = sb.st_size - offset;
        /* Cannot display bytes past end of file */
    } else {           /* No length arg ==> display to end of file */
```




```
    length = sb.st_size - offset;
}
addr = mmap(NULL, length + offset - pa_offset, PROT_READ,
            MAP_PRIVATE, fd, pa_offset);
if (addr == MAP_FAILED)
    handle_error("mmap");
s = write(STDOUT_FILENO, addr + offset - pa_offset, length);
if (s != length) {
    if (s == -1)
        handle_error("write");
    fprintf(stderr, "partial write");
    exit(EXIT_FAILURE);
}
exit(EXIT_SUCCESS);
}
```

See Also

[getpagesize\(2\)](#), [mincore\(2\)](#), [mlock\(2\)](#), [mmap2\(2\)](#), [mprotect\(2\)](#), [mremap\(2\)](#), [msync\(2\)](#), [remap_file_pages\(2\)](#), [setrlimit\(2\)](#), [shmat\(2\)](#), [shm_open\(3\)](#), [shm_overview\(7\)](#)

B.O. Gallmeister, POSIX.4, O'Reilly, pp. 128-129 and 389-391.

Referenced By

[clone\(2\)](#), [explain\(1\)](#), [explain\(3\)](#), [explain_munmap\(3\)](#), [explain_munmap_or_die\(3\)](#), [madvise\(2\)](#), [mlockall\(2\)](#), [mmap_private\(3\)](#), [mmap_read\(3\)](#), [mmap_shared\(3\)](#), [tiffopen\(3\)](#), [zzip_disk_open\(3\)](#)

open(2) - Linux man page



Name

open, creat - open and possibly create a file or device

Synopsis

```
#include <sys/types.h>#include <sys/stat.h>#include <fcntl.h>
int open(const char *pathname, int flags);int open(const char *pathname,
int flags, mode_t mode);int creat(const char *pathname, mode_t mode);
```

Description

Given a *pathname* for a file, **open()** returns a file descriptor, a small, nonnegative integer for use in subsequent system calls (**read(2)**, **write(2)**, **lseek(2)**, **fcntl(2)**, etc.). The file descriptor returned by a successful call will be the lowest-numbered file descriptor not currently open for the process.

By default, the new file descriptor is set to remain open across an **execve(2)** (i.e., the **FD_CLOEXEC** file descriptor flag described in **fcntl(2)** is initially disabled; the **O_CLOEXEC** flag, described below, can be used to change this default). The file offset is set to the beginning of the file (see **lseek(2)**).

A call to **open()** creates a new *open file description*, an entry in the system-wide table of open files. This entry records the file offset and the file status flags (modifiable via the **fcntl(2)** **F_SETFL** operation). A file descriptor is a reference to one of these entries; this reference is unaffected if *pathname* is subsequently removed or modified to refer to a different file. The new open file description is initially not shared with any other process, but sharing may arise via **fork(2)**.

The argument *flags* must include one of the following *access modes*: **O_RDONLY**, **O_WRONLY**, or **O_RDWR**. These request opening the file read-only, write-only, or read/write, respectively.

In addition, zero or more file creation flags and file status flags can be bitwise-or'd in *flags*. The *file* creation flags are **O_CLOEXEC**, **O_CREAT**, **O_DIRECTORY**, **O_EXCL**, **O_NOCTTY**, **O_NOFOLLOW**, **O_TRUNC**, and **O_TTY_INIT**. The *file* status flags are all of the remaining flags listed below. The distinction between these two groups of flags is that the file status flags can be retrieved and (in some cases) modified using **fcntl(2)**. The full list of file creation flags and file status flags is as follows:

O_APPEND

The file is opened in append mode. Before each **write(2)**, the file offset is positioned at the end of the file, as if with **lseek(2)**. **O_APPEND** may lead to corrupted files on



NFS file systems if more than one process appends data to a file at once. This is because NFS does not support appending to a file, so the client kernel has to simulate it, which can't be done without a race condition.

O_ASYNC

Enable signal-driven I/O: generate a signal (**SIGIO** by default, but this can be changed via **fcntl(2)**) when input or output becomes possible on this file descriptor. This feature is only available for terminals, pseudoterminals, sockets, and (since Linux 2.6) pipes and FIFOs. See **fcntl(2)** for further details.

O_CLOEXEC (Since Linux 2.6.23)

Enable the close-on-exec flag for the new file descriptor. Specifying this flag permits a program to avoid additional **fcntl(2)** **F_SETFD** operations to set the **FD_CLOEXEC** flag. Additionally, use of this flag is essential in some multithreaded programs since using a separate **fcntl(2)** **F_SETFD** operation to set the **FD_CLOEXEC** flag does not suffice to avoid race conditions where one thread opens a file descriptor at the same time as another thread does a **fork(2)** plus **execve(2)**.

O_CREAT

If the file does not exist it will be created. The owner (user ID) of the file is set to the effective user ID of the process. The group ownership (group ID) is set either to the effective group ID of the process or to the group ID of the parent directory (depending on file system type and mount options, and the mode of the parent directory, see the mount options *bsdgroups* and *sysvgroups* described in **mount(8)**).

mode specifies the permissions to use in case a new file is created. This argument must be supplied when **O_CREAT** is specified in *flags*; if **O_CREAT** is not specified, then *mode* is ignored. The effective permissions are modified by the process's *umask* in the usual way: The permissions of the created file are $(mode \& \sim umask)$. Note that this mode only applies to future accesses of the newly created file; the **open()** call that creates a read-only file may well return a read/write file descriptor.

The following symbolic constants are provided for *mode*:

S_IRWXU

00700 user (file owner) has read, write and execute permission

S_IRUSR

00400 user has read permission

S_IWUSR

00200 user has write permission

S_IXUSR

00100 user has execute permission

S_IRWXG

00070 group has read, write and execute permission

S_IRGRP

00040 group has read permission

S_IWGRP



00020 group has write permission

S_IXGRP

00010 group has execute permission

S_IRWXO

00007 others have read, write and execute permission

S_IROTH

00004 others have read permission

S_IWOTH

00002 others have write permission

S_IXOTH

00001 others have execute permission

O_DIRECT (Since Linux 2.4.10)

Try to minimize cache effects of the I/O to and from this file. In general this will degrade performance, but it is useful in special situations, such as when applications do their own caching. File I/O is done directly to/from user-space buffers. The **O_DIRECT** flag on its own makes an effort to transfer data synchronously, but does not give the guarantees of the **O_SYNC** flag that data and necessary metadata are transferred. To guarantee synchronous I/O, **O_SYNC** must be used in addition to **O_DIRECT**. See NOTES below for further discussion.

A semantically similar (but deprecated) interface for block devices is described in [**raw**\(8\)](#).

O_DIRECTORY

If *pathname* is not a directory, cause the open to fail. This flag is Linux-specific, and was added in kernel version 2.1.126, to avoid denial-of-service problems if [**opendir**\(3\)](#) is called on a FIFO or tape device, but should not be used outside of the implementation of [**opendir**\(3\)](#).

O_EXCL

Ensure that this call creates the file: if this flag is specified in conjunction with **O_CREAT**, and *pathname* already exists, then **open()** will fail.

When these two flags are specified, symbolic links are not followed: if *pathname* is a symbolic link, then **open()** fails regardless of where the symbolic link points to.

In general, the behavior of **O_EXCL** is undefined if it is used without **O_CREAT**. There is one exception: on Linux 2.6 and later, **O_EXCL** can be used without **O_CREAT** if *pathname* refers to a block device. If the block device is in use by the system (e.g., mounted), **open()** fails with the error **EBUSY**.

On NFS, **O_EXCL** is only supported when using NFSv3 or later on kernel 2.6 or later. In NFS environments where **O_EXCL** support is not provided, programs that rely on it for performing locking tasks will contain a race condition. Portable programs that want



to perform atomic file locking using a lockfile, and need to avoid reliance on NFS support for **O_EXCL**, can create a unique file on the same file system (e.g., incorporating hostname and PID), and use [*link*\(2\)](#) to make a link to the lockfile. If [*link*\(2\)](#) returns 0, the lock is successful. Otherwise, use [*stat*\(2\)](#) on the unique file to check if its link count has increased to 2, in which case the lock is also successful.

O_LARGEFILE

(LFS) Allow files whose sizes cannot be represented in an *off_t* (but can be represented in an *off64_t*) to be opened. The **__LARGEFILE64_SOURCE** macro must be defined (before including *any* header files) in order to obtain this definition. Setting the **__FILE_OFFSET_BITS** feature test macro to 64 (rather than using **O_LARGEFILE**) is the preferred method of accessing large files on 32-bit systems (see [*feature test macros*\(7\)](#)).

O_NOATIME (Since Linux 2.6.8)

Do not update the file last access time (*st_atime* in the inode) when the file is [*read*\(2\)](#). This flag is intended for use by indexing or backup programs, where its use can significantly reduce the amount of disk activity. This flag may not be effective on all file systems. One example is NFS, where the server maintains the access time.

O_NOCTTY

If *pathname* refers to a terminal device--see [*tty*\(4\)](#)--it will not become the process's controlling terminal even if the process does not have one.

O_NOFOLLOW

If *pathname* is a symbolic link, then the open fails. This is a FreeBSD extension, which was added to Linux in version 2.1.126. Symbolic links in earlier components of the *pathname* will still be followed.

O_NONBLOCK or **O_NDELAY**

When possible, the file is opened in nonblocking mode. Neither the **open()** nor any subsequent operations on the file descriptor which is returned will cause the calling process to wait. For the handling of FIFOs (named pipes), see also [*fifo*\(7\)](#). For a discussion of the effect of **O_NONBLOCK** in conjunction with mandatory file locks and with file leases, see [*fcntl*\(2\)](#).

O_SYNC

The file is opened for synchronous I/O. Any [*write*\(2\)](#)s on the resulting file descriptor will block the calling process until the data has been physically written to the underlying hardware. *But see NOTES below.*

O_TRUNC

If the file already exists and is a regular file and the open mode allows writing (i.e., is **O_RDWR** or **O_WRONLY**) it will be truncated to length 0. If the file is a FIFO or terminal device file, the **O_TRUNC** flag is ignored. Otherwise the effect of **O_TRUNC** is unspecified.

Some of these optional flags can be altered using [*fcntl*\(2\)](#) after the file has been opened.

creat() is equivalent to **open()** with *flags* equal to **O_CREAT|O_WRONLY|O_TRUNC**.

Return Value



open() and **creat()** return the new file descriptor, or -1 if an error occurred (in which case, *errno* is set appropriately).

Errors

EACCES

The requested access to the file is not allowed, or search permission is denied for one of the directories in the path prefix of *pathname*, or the file did not exist yet and write access to the parent directory is not allowed. (See also [*path resolution*\(7\)](#).)

EDQUOT

Where **O_CREAT** is specified, the file does not exist, and the user's quota of disk blocks or inodes on the file system has been exhausted.

EEXIST

pathname already exists and **O_CREAT** and **O_EXCL** were used.

EFAULT

pathname points outside your accessible address space.

EFBIG

See **EOVERFLOW**.

EINTR

While blocked waiting to complete an open of a slow device (e.g., a FIFO; see [*fifo*\(7\)](#)), the call was interrupted by a signal handler; see [*signal*\(7\)](#).

EISDIR

pathname refers to a directory and the access requested involved writing (that is, **O_WRONLY** or **O_RDWR** is set).

ELOOP

Too many symbolic links were encountered in resolving *pathname*, or **O_NOFOLLOW** was specified but *pathname* was a symbolic link.

EMFILE

The process already has the maximum number of files open.

ENAMETOOLONG

pathname was too long.

ENFILE

The system limit on the total number of open files has been reached.

ENODEV

pathname refers to a device special file and no corresponding device exists. (This is a Linux kernel bug; in this situation **ENXIO** must be returned.)

ENOENT

O_CREAT is not set and the named file does not exist. Or, a directory component in *pathname* does not exist or is a dangling symbolic link.

ENOMEM

Insufficient kernel memory was available.

ENOSPC



pathname was to be created but the device containing *pathname* has no room for new file.

ENOTDIR

A component used as a directory in *pathname* is not, in fact, a directory, or **O_DIRECTORY** was specified and *pathname* was not a directory.

ENXIO

O_NONBLOCK | **O_WRONLY** is set, the named file is a FIFO and no process has the file open for reading. Or, the file is a device special file and no corresponding device exists.

EOVERFLOW

pathname refers to a regular file that is too large to be opened. The usual scenario here is that an application compiled on a 32-bit platform without - *D_FILE_OFFSET_BITS=64* tried to open a file whose size exceeds $(2^{31}-1)$ bits; see also **O_LARGEFILE** above. This is the error specified by POSIX.1-2001; in kernels before 2.6.24, Linux gave the error **EFBIG** for this case.

EPERM

The **O_NOATIME** flag was specified, but the effective user ID of the caller did not match the owner of the file and the caller was not privileged (**CAP_FOWNER**).

EROFS

pathname refers to a file on a read-only file system and write access was requested.

ETXTBSY

pathname refers to an executable image which is currently being executed and write access was requested.

EWouldBlock

The **O_NONBLOCK** flag was specified, and an incompatible lease was held on the file (see [*fcntl\(2\)*](#)).

Conforming to

SVr4, 4.3BSD, POSIX.1-2001. The **O_DIRECTORY**, **O_NOATIME**, and **O_NOFOLLOW** flags are Linux-specific, and one may need to define **_GNU_SOURCE** (before including *any* header files) to obtain their definitions.

The **O_CLOEXEC** flag is not specified in POSIX.1-2001, but is specified in POSIX.1-2008.

O_DIRECT is not specified in POSIX; one has to define **_GNU_SOURCE** (before including *any* header files) to get its definition.

Notes

Under Linux, the **O_NONBLOCK** flag indicates that one wants to open but does not necessarily have the intention to read or write. This is typically used to open devices in order to get a file descriptor for use with [*ioctl\(2\)*](#).



Unlike the other values that can be specified in *flags*, the *access mode* values **O_RDONLY**, **O_WRONLY**, and **O_RDWR**, do not specify individual bits. Rather, they define the low order two bits of *flags*, and are defined respectively as 0, 1, and 2. In other words, the combination **O_RDONLY | O_WRONLY** is a logical error, and certainly does not have the same meaning as **O_RDWR**. Linux reserves the special, nonstandard access mode 3 (binary 11) in *flags* to mean: check for read and write permission on the file and return a descriptor that can't be used for reading or writing. This nonstandard access mode is used by some Linux drivers to return a descriptor that is only to be used for device-specific **ioctl(2)** operations.

The (undefined) effect of **O_RDONLY | O_TRUNC** varies among implementations. On many systems the file is actually truncated.

There are many infelicities in the protocol underlying NFS, affecting amongst others **O_SYNC** and **O_NDELAY**.

POSIX provides for three different variants of synchronized I/O, corresponding to the flags **O_SYNC**, **O_DSYNC**, and **O_RSYNC**. Currently (2.6.31), Linux only implements **O_SYNC**, but glibc maps **O_DSYNC** and **O_RSYNC** to the same numerical value as **O_SYNC**. Most Linux file systems don't actually implement the POSIX **O_SYNC** semantics, which require all metadata updates of a write to be on disk on returning to user space, but only the **O_DSYNC** semantics, which require only actual file data and metadata necessary to retrieve it to be on disk by the time the system call returns.

Note that **open()** can open device special files, but **creat()** cannot create them; use **mknod(2)** instead.

On NFS file systems with UID mapping enabled, **open()** may return a file descriptor but, for example, **read(2)** requests are denied with **EACCES**. This is because the client performs **open()** by checking the permissions, but UID mapping is performed by the server upon read and write requests.

If the file is newly created, its *st_atime*, *st_ctime*, *st_mtime* fields (respectively, time of last access, time of last status change, and time of last modification; see **stat(2)**) are set to the current time, and so are the *st_ctime* and *st_mtime* fields of the parent directory. Otherwise, if the file is modified because of the **O_TRUNC** flag, its *st_ctime* and *st_mtime* fields are set to the current time.

O_direct

The **O_DIRECT** flag may impose alignment restrictions on the length and address of user-space buffers and the file offset of I/Os. In Linux alignment restrictions vary by file system and kernel version and might be absent entirely. However there is currently no file system-independent interface for an application to discover these restrictions for a given file or file

system. Some file systems provide their own interfaces for doing so, for example the **XFS_IOC_DIOINFO** operation in ***xfstl(3)***.



Under Linux 2.4, transfer sizes, and the alignment of the user buffer and the file offset must all be multiples of the logical block size of the file system. Under Linux 2.6, alignment to 512-byte boundaries suffices.

O_DIRECT I/Os should never be run concurrently with the ***fork(2)*** system call, if the memory buffer is a private mapping (i.e., any mapping created with the ***mmap(2)*** **MAP_PRIVATE** flag; this includes memory allocated on the heap and statically allocated buffers). Any such I/Os, whether submitted via an asynchronous I/O interface or from another thread in the process, should be completed before ***fork(2)*** is called. Failure to do so can result in data corruption and undefined behavior in parent and child processes. This restriction does not apply when the memory buffer for the **O_DIRECT** I/Os was created using ***shmat(2)*** or ***mmap(2)*** with the **MAP_SHARED** flag. Nor does this restriction apply when the memory buffer has been advised as **MADV_DONTFORK** with ***madvise(2)***, ensuring that it will not be available to the child after ***fork(2)***.

The **O_DIRECT** flag was introduced in SGI IRIX, where it has alignment restrictions similar to those of Linux 2.4. IRIX has also a ***fcntl(2)*** call to query appropriate alignments, and sizes. FreeBSD 4.x introduced a flag of the same name, but without alignment restrictions.

O_DIRECT support was added under Linux in kernel version 2.4.10. Older Linux kernels simply ignore this flag. Some file systems may not implement the flag and **open()** will fail with **EINVAL** if it is used.

Applications should avoid mixing **O_DIRECT** and normal I/O to the same file, and especially to overlapping byte regions in the same file. Even when the file system correctly handles the coherency issues in this situation, overall I/O throughput is likely to be slower than using either mode alone. Likewise, applications should avoid mixing ***mmap(2)*** of files with direct I/O to the same files.

The behaviour of **O_DIRECT** with NFS will differ from local file systems. Older kernels, or kernels configured in certain ways, may not support this combination. The NFS protocol does not support passing the flag to the server, so **O_DIRECT** I/O will only bypass the page cache on the client; the server may still cache the I/O. The client asks the server to make the I/O synchronous to preserve the synchronous semantics of **O_DIRECT**. Some servers will perform poorly under these circumstances, especially if the I/O size is small. Some servers may also be configured to lie to clients about the I/O having reached stable storage; this will avoid the performance penalty at some risk to data integrity in the event of server power failure. The Linux NFS client places no alignment restrictions on **O_DIRECT** I/O.

In summary, **O_DIRECT** is a potentially powerful tool that should be used with caution. It is recommended that applications treat use of **O_DIRECT** as a performance option which

is disabled by default.



"The thing that has always disturbed me about O_DIRECT is that the whole interface is just stupid, and was probably designed by a deranged monkey on some serious mind-controlling substances."--Linus

Bugs

Currently, it is not possible to enable signal-driven I/O by specifying **O_ASYNC** when calling **open()**; use **fcntl(2)** to enable this flag.

See Also

chmod(2), **chown(2)**, **close(2)**, **dup(2)**, **fcntl(2)**, **link(2)**, **lseek(2)**, **mknod(2)**, **mmap(2)**, **mount(2)**, **openat(2)**, **read(2)**, **socket(2)**, **stat(2)**, **umask(2)**, **unlink(2)**, **write(2)**, **fopen(3)**, **fifo(7)**, **path_resolution(7)**, **symlink(7)**

Referenced By

0install(1), **0launch(1)**, **accept4(2)**, **access(2)**, **acl(5)**, **archive_read(3)**, **archive_write(3)**, **capabilities(7)**, **catopen(3)**, **clone(2)**, **config(3)**, **cpuset(7)**, **dbopen(3)**, **ddpt(8)**, **dirfd(3)**, **dup2(2)**, **epoll(7)**, **epoll_create(2)**, **euidaccess(3)**, **eventfd(2)**, **explain(1)**, **explain(3)**, **explain_lca2010(1)**, **explain_open(3)**, **explain_open_or_die(3)**, **explain_output(3)**, **fakechroot(1)**, **ferror(3)**, **fifo(4)**, **flock(2)**, **flopen(3)**, **fsync(2)**, **fts(3)**, **gdbm(3)**, **getcwd(2)**, **getdirenties(3)**, **getdtablesize(2)**, **getdtablesize(3)**, **getfilecon(3)**, **getpt(3)**, **getrlimit(2)**, **getutent(3)**, **getxattr(2)**, **grantpt(3)**, **hal-disable-polling(1)**, **hylafax-log(5)**, **inotify_init(2)**, **ioprio_set(2)**, **kcmp(2)**, **ksh(1)**, **lam_rfposix(2)**, **lam_rfstate(2)**, **lam_rtrget(2)**, **lamf_rfclose(2)**, **listxattr(2)**, **lp(4)**, **lsof(8)**, **mkfifo(3)**, **mksh(1)**, **mkstemp(3)**, **mq_open(3)**, **mq_overview(7)**, **msgget(2)**, **ncl_cgm(3)**, **nfslogsum(8)**, **open_append(3)**, **open_by_handle(3)**, **open_excl(3)**, **open_read(3)**, **open_rw(3)**, **open_trunc(3)**, **open_write(3)**, **pathconf(3)**, **perf_event_open(2)**, **perlfunc(1)**, **perlomentut(1)**, **pidfile(3)**, **pipe(2)**, **pipe(7)**, **popen(3)**, **posix(3)**, **posix_openpt(3)**, **proc(5)**, **pthread_setname_np(3)**, **pty(7)**, **recv(2)**, **remove(3)**, **removexattr(2)**, **rmt(8)**, **samefile(1)**, **sem_open(3)**, **semget(2)**, **sendfile(2)**, **setfilecon(3)**, **setxattr(2)**, **shm_open(3)**, **shm_overview(7)**, **shmget(2)**, **signalfd(2)**, **st(4)**, **stdin(3)**, **stdio(3)**, **symlink(2)**, **tar_append_file(3)**, **tar_open(3)**, **tempnam(3)**, **tiffquery(3)**, **timerfd_create(2)**, **tmpnam(3)**, **truncate(2)**, **tty_ioctl(4)**, **uselib(2)**, **vploaddrawvolume(3)**, **vpsetcallback(3)**, **vpstoreraawvolume(3)**, **wipe(1)**, **writew(2)**, **xfs_io(8)**, **zvbi-chains(1)**

pipe(2) - Linux man page



Name

pipe, pipe2 - create pipe

Synopsis

```
#include <unistd.h>
int pipe(int pipefd[2]);
#define _GNU_SOURCE          /* See feature\_test\_macros\(7\) */ #include
<fcntl.h>                  /* Obtain O_* constant definitions */ #include <unistd.h>
int pipe2(int pipefd[2], int flags);
```

Description

pipe() creates a pipe, a unidirectional data channel that can be used for interprocess communication. The array *pipefd* is used to return two file descriptors referring to the ends of the pipe. *pipefd[0]* refers to the read end of the pipe. *pipefd[1]* refers to the write end of the pipe. Data written to the write end of the pipe is buffered by the kernel until it is read from the read end of the pipe. For further details, see [pipe\(7\)](#).

If *flags* is 0, then **pipe2()** is the same as **pipe()**. The following values can be bitwise ORed in *flags* to obtain different behavior:

O_NONBLOCK

Set the **O_NONBLOCK** file status flag on the two new open file descriptions. Using this flag saves extra calls to [fcntl\(2\)](#) to achieve the same result.

O_CLOEXEC

Set the close-on-exec (**FD_CLOEXEC**) flag on the two new file descriptors. See the description of the same flag in [open\(2\)](#) for reasons why this may be useful.

Return Value

On success, zero is returned. On error, -1 is returned, and *errno* is set appropriately.

Errors

EFAULT

pipefd is not valid.

EINVAL

(**pipe2()**) Invalid value in *flags*.

EMFILE

Too many file descriptors are in use by the process.

ENFILE



The system limit on the total number of open files has been reached.

Versions

pipe2() was added to Linux in version 2.6.27; glibc support is available starting with version 2.9.

Conforming to

pipe(): POSIX.1-2001.

pipe2() is Linux-specific.

Example

The following program creates a pipe, and then **fork(2)**s to create a child process; the child inherits a duplicate set of file descriptors that refer to the same pipe. After the **fork(2)**, each process closes the descriptors that it doesn't need for the pipe (see **pipe(7)**). The parent then writes the string contained in the program's command-line argument to the pipe, and the child reads this string a byte at a time from the pipe and echoes it on standard output.

```
#include <sys/wait.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
int
main(int argc, char *argv[])
{
    int pipefd[2];
    pid_t cpid;
    char buf;
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <string>\n", argv[0]);
        exit(EXIT_FAILURE);
    }
    if (pipe(pipefd) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }
    cpid = fork();
    if (cpid == -1) {
        perror("fork");
        exit(EXIT_FAILURE);
    }
```



```
}
if (cpid == 0) {    /* Child reads from pipe */
    close(pipefd[1]);    /* Close unused write end */
    while (read(pipefd[0], &buf, 1) > 0)
        write(STDOUT_FILENO, &buf, 1);
    write(STDOUT_FILENO, "\n", 1);
    close(pipefd[0]);
    _exit(EXIT_SUCCESS);
} else {           /* Parent writes argv[1] to pipe */
    close(pipefd[0]);    /* Close unused read end */
    write(pipefd[1], argv[1], strlen(argv[1]));
    close(pipefd[1]);    /* Reader will see EOF */
    wait(NULL);          /* Wait for child */
    exit(EXIT_SUCCESS);
}
}
```

See Also

[**fork**](#)(2), [**read**](#)(2), [**socketpair**](#)(2), [**write**](#)(2), [**popen**](#)(3), [**pipe**](#)(7)

Referenced By

[**capabilities**](#)(7), [**cs**](#)(1), [**event**](#)(3), [**eventfd**](#)(2), [**eventtest**](#)(6), [**explain**](#)(1), [**explain**](#)(3), [**explain_pipe**](#)(3), [**explain_pipe_or_die**](#)(3), [**fifo**](#)(4), [**fifo**](#)(7), [**getrlimit**](#)(2), [**iv_event_raw**](#)(3), [**ksh**](#)(1), [**ksh93**](#)(1), [**man-pages**](#)(7), [**mksh**](#)(1), [**pmdaconnect**](#)(3), [**sec**](#)(1), [**shmux**](#)(1), [**stat**](#)(2)

read(2) - Linux man page



Name

read - read from a file descriptor

Synopsis

```
#include <unistd.h>
ssize_t read(int fd, void *buf, size_t count);
```

Description

read() attempts to read up to *count* bytes from file descriptor *fd* into the buffer starting at *buf*.

On files that support seeking, the read operation commences at the current file offset, and the file offset is incremented by the number of bytes read. If the current file offset is at or past the end of file, no bytes are read, and **read()** returns zero.

If *count* is zero, **read()** may detect the errors described below. In the absence of any errors, or if **read()** does not check for errors, a **read()** with a *count* of 0 returns zero and has no other effects.

If *count* is greater than **SSIZE_MAX**, the result is unspecified.

Return Value

On success, the number of bytes read is returned (zero indicates end of file), and the file position is advanced by this number. It is not an error if this number is smaller than the number of bytes requested; this may happen for example because fewer bytes are actually available right now (maybe because we were close to end-of-file, or because we are reading from a pipe, or from a terminal), or because **read()** was interrupted by a signal. On error, -1 is returned, and *errno* is set appropriately. In this case it is left unspecified whether the file position (if any) changes.

Errors

EAGAIN

The file descriptor *fd* refers to a file other than a socket and has been marked nonblocking (**O_NONBLOCK**), and the read would block.

EAGAIN or EWOULDBLOCK

The file descriptor *fd* refers to a socket and has been marked nonblocking (**O_NONBLOCK**), and the read would block. POSIX.1-2001 allows either error to be returned for this case, and does not require these constants to have the same value, so a portable application should check for both possibilities.

**EBADF**

fd is not a valid file descriptor or is not open for reading.

EFAULT

buf is outside your accessible address space.

EINTR

The call was interrupted by a signal before any data was read; see [**signal**\(7\)](#).

EINVAL

fd is attached to an object which is unsuitable for reading; or the file was opened with the **O_DIRECT** flag, and either the address specified in *buf*, the value specified in *count*, or the current file offset is not suitably aligned.

EINVAL

fd was created via a call to [**timerfd_create**\(2\)](#) and the wrong size buffer was given to **read()**; see [**timerfd_create**\(2\)](#) for further information.

EIO

I/O error. This will happen for example when the process is in a background process group, tries to read from its controlling terminal, and either it is ignoring or blocking **SIGTTIN** or its process group is orphaned. It may also occur when there is a low-level I/O error while reading from a disk or tape.

EISDIR

fd refers to a directory.

Other errors may occur, depending on the object connected to *fd*. POSIX allows a **read()** that is interrupted after reading some data to return -1 (with *errno* set to **EINTR**) or to return the number of bytes already read.

Conforming to

SVr4, 4.3BSD, POSIX.1-2001.

Notes

On NFS file systems, reading small amounts of data will only update the timestamp the first time, subsequent calls may not do so. This is caused by client side attribute caching, because most if not all NFS clients leave *st_atime* (last file access time) updates to the server and client side reads satisfied from the client's cache will not cause *st_atime* updates on the server as there are no server side reads. UNIX semantics can be obtained by disabling client side attribute caching, but in most situations this will substantially increase server load and decrease performance.

See Also

[**close**\(2\)](#), [**fcntl**\(2\)](#), [**ioctl**\(2\)](#), [**lseek**\(2\)](#), [**open**\(2\)](#), [**pread**\(2\)](#), [**readdir**\(2\)](#), [**readlink**\(2\)](#), [**readv**\(2\)](#), [**select**\(2\)](#), [**write**\(2\)](#), [**fread**\(3\)](#)

Referenced By



[aio](#)(7), [aio_error](#)(3), [aio_read](#)(3), [aio_return](#)(3), [archive_read](#)(3), [cpuset](#)(7), [c](#)
[dbopen](#)(3), [dsp56k](#)(4), [epoll](#)(4), [epoll](#)(7), [epoll_ctl](#)(2), [eventfd](#)(2), [explain](#)(1),
[explain](#)(3), [explain_lca2010](#)(1), [explain_open_or_die](#)(3), [explain_read](#)(3),
[explain_read_or_die](#)(3), [fgetln](#)(3), [fgets](#)(3), [fio](#)(1), [fopen](#)(3), [getline](#)(3), [grep](#)(1),
[hosts_options](#)(5), [htop](#)(1), [hylafax-log](#)(5), [inotify](#)(7), [inotify_add_watch](#)(2),
[io_canread](#)(3), [iostat](#)(1), [iv_fd_pump_pump](#)(3), [lam_rfposix](#)(2), [lamf_rfwrite](#)(2),
[libarchive](#)(3), [libexpect](#)(3), [libssh2_sftp_read](#)(3), [mkfifo](#)(3), [mpool](#)(3), [ncl_cgm](#)(3),
[nfslogsum](#)(8), [perf_event_open](#)(2), [perfmonctl](#)(2), [perlfunc](#)(1), [perlpacktut](#)(1),
[pidfile](#)(3), [pipe](#)(2), [pipe](#)(7), [pipethrough](#)(3), [prctl](#)(2), [proc](#)(5), [pth](#)(3), [ptrace](#)(2),
[readahead](#)(2), [readdir](#)(3), [recv](#)(2), [rmt](#)(8), [rtc](#)(4), [rtime](#)(3), [scrub](#)(1), [select_tut](#)(2),
[sendfile](#)(2), [setpgrp](#)(2), [sg_dd](#)(8), [signalfd](#)(2), [socket](#)(2), [socket](#)(7), [socketpair](#)(2),
[spew](#)(1), [spufs](#)(2), [spufs](#)(7), [st](#)(4), [stat](#)(2), [stdin](#)(3), [stdio](#)(3), [strace](#)(1),
[tar_append_file](#)(3), [tar_block_write](#)(3), [tcflush](#)(3), [tty_ioctl](#)(4), [vpsetcallback](#)(3),
[vrb](#)(3), [vrb_read](#)(3), [vrb_read_min](#)(3), [wgetch](#)(3), [writev](#)(2), [x25](#)(7), [xdr](#)(3),
[xfs_io](#)(8), [xfsctl](#)(3), [zero](#)(4), [zvbi-chains](#)(1), [zzip_freopen](#)(3)

sbrk(2) - Linux man page



Name

brk, sbrk - change data segment size

Synopsis

#include <[unistd.h](#)>

int brk(**void** **addr*);

void *sbrk(**intptr_t** *increment*);

Feature Test Macro Requirements for glibc (see [feature_test_macros\(7\)](#)):

brk(), **sbrk()**:

Since glibc 2.12:

```
_BSD_SOURCE || _SVID_SOURCE ||  
    (_XOPEN_SOURCE >= 500 ||  
     _XOPEN_SOURCE && _XOPEN_SOURCE_EXTENDED) &&  
    !(_POSIX_C_SOURCE >= 200112L || _XOPEN_SOURCE >= 600)
```

Before glibc 2.12:

```
_BSD_SOURCE || _SVID_SOURCE || _XOPEN_SOURCE >= 500 || _XOPEN_SOURCE  
&& _XOPEN_SOURCE_EXTENDED
```

Description

brk() and **sbrk()** change the location of the *program break*, which defines the end of the process's data segment (i.e., the program break is the first location after the end of the uninitialized data segment). Increasing the program break has the effect of allocating memory to the process; decreasing the break deallocates memory.

brk() sets the end of the data segment to the value specified by *addr*, when that value is reasonable, the system has enough memory, and the process does not exceed its maximum data size (see [setrlimit\(2\)](#)).

sbrk() increments the program's data space by *increment* bytes. Calling **sbrk()** with an *increment* of 0 can be used to find the current location of the program break.

Return Value

On success, **brk()** returns zero. On error, -1 is returned, and *errno* is set to **ENOMEM**. (But see *Linux Notes* below.)

On success, **sbrk()** returns the previous program break. (If the break was increased, this value is a pointer to the start of the newly allocated memory). On error, *(void *)* returned, and *errno* is set to **ENOMEM**.



Conforming To

4.3BSD; SUSv1, marked LEGACY in SUSv2, removed in POSIX.1-2001.

Notes

Avoid using **brk()** and **sbrk()**: the **malloc(3)** memory allocation package is the portable and comfortable way of allocating memory.

Various systems use various types for the argument of **sbrk()**. Common are *int*, *ssize_t*, *ptrdiff_t*, *intptr_t*.

Linux notes

The return value described above for **brk()** is the behavior provided by the glibc wrapper function for the Linux **brk()** system call. (On most other implementations, the return value from **brk()** is the same; this return value was also specified in SUSv2.) However, the actual Linux system call returns the new program break on success. On failure, the system call returns the current break. The glibc wrapper function does some work (i.e., checks whether the new break is less than *addr*) to provide the 0 and -1 return values described above.

On Linux, **sbrk()** is implemented as a library function that uses the **brk()** system call, and does some internal bookkeeping so that it can return the old break value.

See Also

execve(2), **getrlimit(2)**, **end(3)**, **malloc(3)**

Referenced By

csh(1), **jemalloc(3)**, **malloc_hook(3)**, **malloc_trim(3)**, **mallopt(3)**, **mlockall(2)**, **mremap(2)**, **perldebguts(1)**, **sstk(2)**, **unbound-control(8)**

wait(2) - Linux man page



Name

wait, waitpid, waitid - wait for process to change state

Synopsis

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
pid_t wait(int *status);
```

```
pid_t waitpid(pid_t pid, int *status, int options);
```

```
int waitid(idtype_t idtype, id_t id, siginfo_t *infop, int options);
```

Feature Test Macro Requirements for glibc (see [feature_test_macros\(7\)](#)):

waitid():

```
_SVID_SOURCE || _XOPEN_SOURCE >= 500 || _XOPEN_SOURCE &&  
_XOPEN_SOURCE_EXTENDED  
|| /* Since glibc 2.12: */ _POSIX_C_SOURCE >= 200809L
```

Description

All of these system calls are used to wait for state changes in a child of the calling process, and obtain information about the child whose state has changed. A state change is considered to be: the child terminated; the child was stopped by a signal; or the child was resumed by a signal. In the case of a terminated child, performing a wait allows the system to release the resources associated with the child; if a wait is not performed, then the terminated child remains in a "zombie" state (see NOTES below).

If a child has already changed state, then these calls return immediately. Otherwise they block until either a child changes state or a signal handler interrupts the call (assuming that system calls are not automatically restarted using the **SA_RESTART** flag of [sigaction\(2\)](#)). In the remainder of this page, a child whose state has changed and which has not yet been waited upon by one of these system calls is termed *waitable*.

wait() and waitpid()

The **wait()** system call suspends execution of the calling process until one of its children terminates. The call *wait(&status)* is equivalent to:

```
waitpid(-1, &status, 0);
```



The **waitpid()** system call suspends execution of the calling process until a child specified by *pid* argument has changed state. By default, **waitpid()** waits only for terminated children, but this behavior is modifiable via the *options* argument, as described below.

The value of *pid* can be:

< -1

meaning wait for any child process whose process group ID is equal to the absolute value of *pid*.

-1

meaning wait for any child process.

0

meaning wait for any child process whose process group ID is equal to that of the calling process.

> 0

meaning wait for the child whose process ID is equal to the value of *pid*.

The value of *options* is an OR of zero or more of the following constants:

WNOHANG

return immediately if no child has exited.

WUNTRACED

also return if a child has stopped (but not traced via **ptrace(2)**). Status for *traced* children which have stopped is provided even if this option is not specified.

WCONTINUED (since Linux 2.6.10)

also return if a stopped child has been resumed by delivery of **SIGCONT**.

(For Linux-only options, see below.)

If *status* is not NULL, **wait()** and **waitpid()** store status information in the *int* to which it points. This integer can be inspected with the following macros (which take the integer itself as an argument, not a pointer to it, as is done in **wait()** and **waitpid()**):

WIFEXITED(status)

returns true if the child terminated normally, that is, by calling **exit(3)** or **_exit(2)**, or by returning from **main()**.

WEXITSTATUS(status)

returns the exit status of the child. This consists of the least significant 8 bits of the *status* argument that the child specified in a call to **exit(3)** or **_exit(2)** or as the



argument for a return statement in `main()`. This macro should only be employed if **WIFEXITED** returned true.

WIFSIGNALED(*status*)

returns true if the child process was terminated by a signal.

WTERMSIG(*status*)

returns the number of the signal that caused the child process to terminate. This macro should only be employed if **WIFSIGNALED** returned true.

WCOREDUMP(*status*)

returns true if the child produced a core dump. This macro should only be employed if **WIFSIGNALED** returned true. This macro is not specified in POSIX.1-2001 and is not available on some UNIX implementations (e.g., AIX, SunOS). Only use this enclosed in `#ifdef WCOREDUMP ... #endif`.

WIFSTOPPED(*status*)

returns true if the child process was stopped by delivery of a signal; this is only possible if the call was done using **WUNTRACED** or when the child is being traced (see **ptrace**(2)).

WSTOPSIG(*status*)

returns the number of the signal which caused the child to stop. This macro should only be employed if **WIFSTOPPED** returned true.

WIFCONTINUED(*status*)

(since Linux 2.6.10) returns true if the child process was resumed by delivery of **SIGCONT**.

waitid()

The **waitid()** system call (available since Linux 2.6.9) provides more precise control over which child state changes to wait for.

The *idtype* and *id* arguments select the child(ren) to wait for, as follows:

idtype == **P_PID**

Wait for the child whose process ID matches *id*.

idtype == **P_PGID**

Wait for any child whose process group ID matches *id*.

idtype == **P_ALL**

Wait for any child; *id* is ignored.

The child state changes to wait for are specified by ORing one or more of the following flags in *options*:

WEXITED

Wait for children that have terminated.

WSTOPPED

Wait for children that have been stopped by delivery of a signal.

WCONTINUED



Wait for (previously stopped) children that have been resumed by delivery of **SIGCONT**. The following flags may additionally be ORed in *options*:

WNOHANG

As for **waitpid()**.

WNOWAIT

Leave the child in a waitable state; a later wait call can be used to again retrieve the child status information.

Upon successful return, **waitid()** fills in the following fields of the *siginfo_t* structure pointed to by *infop*:

si_pid

The process ID of the child.

si_uid

The real user ID of the child. (This field is not set on most other implementations.)

si_signo

Always set to **SIGCHLD**.

si_status

Either the exit status of the child, as given to **_exit(2)** (or **exit(3)**), or the signal that caused the child to terminate, stop, or continue. The *si_code* field can be used to determine how to interpret this field.

si_code

Set to one of: **CLD_EXITED** (child called **_exit(2)**); **CLD_KILLED** (child killed by signal); **CLD_DUMPED** (child killed by signal, and dumped core); **CLD_STOPPED** (child stopped by signal); **CLD_TRAPPED** (traced child has trapped); or **CLD_CONTINUED** (child continued by **SIGCONT**).

If **WNOHANG** was specified in *options* and there were no children in a waitable state, then **waitid()** returns 0 immediately and the state of the *siginfo_t* structure pointed to by *infop* is unspecified. To distinguish this case from that where a child was in a waitable state, zero out the *si_pid* field before the call and check for a nonzero value in this field after the call returns.

Return Value



wait(): on success, returns the process ID of the terminated child; on error, -1 is returned.

waitpid(): on success, returns the process ID of the child whose state has changed; if **WNOHANG** was specified and one or more child(ren) specified by *pid* exist, but have not yet changed state, then 0 is returned. On error, -1 is returned.

waitid(): returns 0 on success or if **WNOHANG** was specified and no child(ren) specified by *id* has yet changed state; on error, -1 is returned. Each of these calls sets *errno* to an appropriate value in the case of an error.

Errors

ECHILD

(for **wait()**) The calling process does not have any unwaited-for children.

ECHILD

(for **waitpid()** or **waitid()**) The process specified by *pid* (**waitpid()**) or *idtype* and *id* (**waitid()**) does not exist or is not a child of the calling process. (This can happen for one's own child if the action for **SIGCHLD** is set to **SIG_IGN**. See also the *Linux Notes* section about threads.)

EINTR

WNOHANG was not set and an unblocked signal or a **SIGCHLD** was caught; see [signal\(7\)](#).

EINVAL

The *options* argument was invalid.

Conforming To

SVr4, 4.3BSD, POSIX.1-2001.

Notes

A child that terminates, but has not been waited for becomes a "zombie". The kernel maintains a minimal set of information about the zombie process (PID, termination status, resource usage information) in order to allow the parent to later perform a wait to obtain information about the child. As long as a zombie is not removed from the system via a wait, it will consume a slot in the kernel process table, and if this table fills, it will not be possible to create further processes. If a parent process terminates, then its "zombie" children (if any) are adopted by [init\(8\)](#), which automatically performs a wait to remove the zombies.



POSIX.1-2001 specifies that if the disposition of **SIGCHLD** is set to **SIG_IGN** or the **SA_NOCLDWAIT** flag is set for **SIGCHLD** (see [sigaction\(2\)](#)), then children that terminate do not become zombies and a call to **wait()** or **waitpid()** will block until all children have terminated, and then fail with *errno* set to **ECHILD**. (The original POSIX standard left the behavior of setting **SIGCHLD** to **SIG_IGN** unspecified. Note that even though the default disposition of **SIGCHLD** is "ignore", explicitly setting the disposition to **SIG_IGN** results in different treatment of zombie process children.) Linux 2.6 conforms to this specification. However, Linux 2.4 (and earlier) does not: if a **wait()** or **waitpid()** call is made while **SIGCHLD** is being ignored, the call behaves just as though **SIGCHLD** were not being ignored, that is, the call blocks until the next child terminates and then returns the process ID and status of that child.

Linux notes

In the Linux kernel, a kernel-scheduled thread is not a distinct construct from a process. Instead, a thread is simply a process that is created using the Linux-unique [clone\(2\)](#) system call; other routines such as the portable [pthread_create\(3\)](#) call are implemented using [clone\(2\)](#). Before Linux 2.4, a thread was just a special case of a process, and as a consequence one thread could not wait on the children of another thread, even when the latter belongs to the same thread group. However, POSIX prescribes such functionality, and since Linux 2.4 a thread can, and by default will, wait on children of other threads in the same thread group.

The following Linux-specific *options* are for use with children created using [clone\(2\)](#); they cannot be used with **waitid()**:

__WCLONE

Wait for "clone" children only. If omitted then wait for "non-clone" children only. (A "clone" child is one which delivers no signal, or a signal other than **SIGCHLD** to its parent upon termination.) This option is ignored if **__WALL** is also specified.

__WALL (since Linux 2.4)

Wait for all children, regardless of type ("clone" or "non-clone").

__WNOTHREAD (since Linux 2.4)

Do not wait for children of other threads in the same thread group. This was the default before Linux 2.4.

Bugs

According to POSIX.1-2008, an application calling **waitid()** must ensure that *info* points to a *siginfo_t* structure (i.e., that it is a non-NULL pointer). On Linux, if *info* is NULL, **waitid()** succeeds, and returns the process ID of the waited-for child. Applications should avoid relying on this inconsistent, nonstandard, and unnecessary feature.

Example

The following program demonstrates the use of **fork(2)** and **waitpid()**. The program creates a child process. If no command-line argument is supplied to the program, the child suspends its execution using **pause(2)**, to allow the user to send signals to the child. Otherwise, if a command-line argument is supplied, then the child exits immediately, using the integer supplied on the command line as the exit status. The parent process executes a loop that monitors the child using **waitpid()**, and uses the **W*()** macros described above to analyze the wait status value.



The following shell session demonstrates the use of the program:

```
$ ./a.out &
Child PID is 32360
[1] 32359
$ kill -STOP 32360
stopped by signal 19
$ kill -CONT 32360
continued
$ kill -TERM 32360
killed by signal 15
[1]+  Done                  ./a.out
$
```

Program source

```
#include <sys/wait.h>
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>

int
main(int argc, char *argv[])
{
    pid_t cpid, w;
    int status;

    cpid = fork();
    if (cpid == -1) {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (cpid == 0) {
        /* Code executed by child */
        printf("Child PID is %ld\n", (long) getpid());
        if (argc == 1)
```




```

        pause();                                /* Wait for signals */
        _exit(atoi(argv[1]));

    } else {                                    /* Code executed by parent */
        do {
            w = waitpid(cpid, &status, WUNTRACED | WCONTINUED);
            if (w == -1) {
                perror("waitpid");
                exit(EXIT_FAILURE);
            }

            if (WIFEXITED(status)) {
                printf("exited, status=%d\n", WEXITSTATUS(status));
            } else if (WIFSIGNALED(status)) {
                printf("killed by signal %d\n", WTERMSIG(status));
            } else if (WIFSTOPPED(status)) {
                printf("stopped by signal %d\n", WSTOPSIG(status));
            } else if (WIFCONTINUED(status)) {
                printf("continued\n");
            }
        } while (!WIFEXITED(status) && !WIFSIGNALED(status));
        exit(EXIT_SUCCESS);
    }
}

```

See Also

[exit\(2\)](#), [clone\(2\)](#), [fork\(2\)](#), [kill\(2\)](#), [ptrace\(2\)](#), [sigaction\(2\)](#), [signal\(2\)](#), [wait4\(2\)](#), [pthread_create\(3\)](#), [credentials\(7\)](#), [signal\(7\)](#)

Referenced By

[acct\(5\)](#), [clock\(3\)](#), [csh\(1\)](#), [exit\(2\)](#), [explain\(1\)](#), [explain\(3\)](#), [explain_lca2010\(1\)](#), [explain_wait\(3\)](#), [explain_wait_or_die\(3\)](#), [filter_wait\(3\)](#), [getrusage\(2\)](#), [ibv_fork_init\(3\)](#), [intro\(1\)](#), [ksh\(1\)](#), [ksh93\(1\)](#), [man-pages\(7\)](#), [mksh\(1\)](#), [nal_connection_new\(2\)](#), [perlfunc\(1\)](#), [perlglossary\(1\)](#), [perlvar\(1\)](#), [prctl\(2\)](#), [pthread_exit\(3\)](#), [pthreads\(7\)](#), [shellout\(3\)](#), [strace\(1\)](#), [sudo_plugin\(8\)](#), [system\(3\)](#), [times\(2\)](#), [vfork\(2\)](#)

write(2) - Linux man page

Name

write - write to a file descriptor

Synopsis

#include <**unistd.h**>

ssize_t write(int *fd*, const void **buf*, **size_t** *count*);

Description

write() writes up to *count* bytes from the buffer pointed *buf* to the file referred to by the file descriptor *fd*.

The number of bytes written may be less than *count* if, for example, there is insufficient space on the underlying physical medium, or the **RLIMIT_FSIZE** resource limit is encountered (see **setrlimit(2)**), or the call was interrupted by a signal handler after having written less than *count* bytes. (See also **pipe(7)**.)

For a seekable file (i.e., one to which **lseek(2)** may be applied, for example, a regular file), writing takes place at the current file offset, and the file offset is incremented by the number of bytes actually written. If the file was **open(2)**ed with **O_APPEND**, the file offset is first set to the end of the file before writing. The adjustment of the file offset and the write operation are performed as an atomic step.



POSIX requires that a **read(2)** which can be proved to occur after a **write()** has returned returns the new data. Note that not all file systems are POSIX conforming.

Return Value

On success, the number of bytes written is returned (zero indicates nothing was written). On error, -1 is returned, and *errno* is set appropriately.

If *count* is zero and *fd* refers to a regular file, then **write()** may return a failure status if one of the errors below is detected. If no errors are detected, 0 will be returned without causing any other effect. If *count* is zero and *fd* refers to a file other than a regular file, the results are not specified.

Errors

EAGAIN

The file descriptor *fd* refers to a file other than a socket and has been marked nonblocking (**O_NONBLOCK**), and the write would block.

EAGAIN or EWOULDBLOCK

The file descriptor *fd* refers to a socket and has been marked nonblocking (**O_NONBLOCK**), and the write would block. POSIX.1-2001 allows either error to be returned for this case, and does not require these constants to have the same value, so a portable application should check for both possibilities.

EBADF

fd is not a valid file descriptor or is not open for writing.

EDESTADDRREQ

fd refers to a datagram socket for which a peer address has not been set using **connect(2)**.

EDQUOT

The user's quota of disk blocks on the file system containing the file referred to by *fd* has been exhausted.

EFAULT

buf is outside your accessible address space.

EFBIG

An attempt was made to write a file that exceeds the implementation-defined maximum file size or the process's file size limit, or to write at a position past the maximum allowed offset.

EINTR

The call was interrupted by a signal before any data was written; see **signal(7)**.

EINVAL

fd is attached to an object which is unsuitable for writing; or the file was opened the **O_DIRECT** flag, and either the address specified in *buf*, the value specified in *count*, or the current file offset is not suitably aligned.

EIO

A low-level I/O error occurred while modifying the inode.

ENOSPC

The device containing the file referred to by *fd* has no room for the data.

EPIPE

fd is connected to a pipe or socket whose reading end is closed. When this happens the writing process will also receive a **SIGPIPE** signal. (Thus, the write return value is seen only if the program catches, blocks or ignores this signal.)

Other errors may occur, depending on the object connected to *fd*.

Conforming to

SVr4, 4.3BSD, POSIX.1-2001.

Under SVr4 a write may be interrupted and return **EINTR** at any point, not just before any data is written.

Notes

A successful return from **write()** does not make any guarantee that data has been committed to disk. In fact, on some buggy implementations, it does not even guarantee that space has successfully been reserved for the data. The only way to be sure is to call **fsync(2)** after you are done writing all your data.

If a **write()** is interrupted by a signal handler before any bytes are written, then the call fails with the error **EINTR**; if it is interrupted after at least one byte has been written, the call succeeds, and returns the number of bytes written.

See Also

close(2), fcntl(2), fsync(2), ioctl(2), lseek(2), open(2), pwrite(2), read(2), select(2), writew(2), fwrite(3)

Referenced By

aio(7), aio_error(3), aio_return(3), aio_write(3), archive_write(3), cpuset(7), dbopen(3), dsp56k(4), epoll(4), epoll(7), epoll_ctl(2), eventfd(2), explain(1), explain(3), explain_open_or_die(3), explain_write(3), explain_write_or_die(3), fclose(3), fflush(3), fgets(3), fio(1), fopen(3), fread(3), getpeername(2), getrlimit(2), hosts_options(5), http(1), im_start_one(3), innxbatch(8), innxmit(8), io_canwrite(3), iostat(1), iv_fd_pump_pump(3), lam_rfposix(2), lamf_rfwrite(2), libexpect(3), libraptor(3), mcprint(3), mkfifo(3), mmap(2), mpool(3), ncl_cgm , nfslogsum(8), nttcp(1), perlfaq5(1), perlfunc(1), pidfile(3), pipe(2), pipethrough(3), pmcontextconnectto(3), prctl(2), pread(2), pth(3), putc(3), rmt(8), scrub(1), select_tut(2), send(2), sendfile(2), sg_dd(8), socket(2), socket(7), socketpair(2), spax(1), spew(1), spufs(2), spufs(7), st(4), stat(2), stdio(3), strace(1), tar_block_write(3), tar_extract_regfile(3), tevent_queue_tutorial(3), udp(7), vpsetcallback(3), vrbl(3), vrbl_write(3), vrbl_write_min(3), x25(7), xdr(3), xfs_io(8), xfsctl(3), zfs(8)

atoi(3) - Linux man page

Name

atoi, atol, atoll, atoll - convert a string to an integer

Synopsis

```
#include <stdlib.h>
```

```
int atoi(const char *nptr);  
long atol(const char *nptr);  
long long atoll(const char *nptr);  
long long atoll(const char *nptr);
```

Feature Test Macro Requirements for glibc (see [feature_test_macros\(7\)](#)):

atoll():

```
_BSD_SOURCE || _SVID_SOURCE || _XOPEN_SOURCE >= 600 || _ISOC99_SOURCE  
|| _POSIX_C_SOURCE >= 200112L;  
or cc -std=c99
```

Description



The **atoi()** function converts the initial portion of the string pointed to by *nptr* to *int*. The behavior is the same as

```
strtol(nptr, NULL, 10);
```

except that **atoi()** does not detect errors.

The **atol()** and **atoll()** functions behave the same as **atoi()**, except that they convert the initial portion of the string to their return type of *long* or *long long*. **atoll()** is an obsolete name for **atoll()**.

Return Value

The converted value.

Conforming To

SVr4, POSIX.1-2001, 4.3BSD, C99. C89 and POSIX.1-1996 include the functions **atoi()** and **atol()** only. **atoll()** is a GNU extension.

Notes

The nonstandard **atoq()** function is not present in libc 4.6.27 or glibc 2, but is present in libc5 and libc 4.7 (though only as an inline function in [*<stdlib.h>*](#) until libc 5.4.44). The **atoll()** function is present in glibc 2 since version 2.0.2, but not in libc4 or libc5.

See Also

[**atof\(3\)**](#), [**strtod\(3\)**](#), [**strtol\(3\)**](#), [**strtoul\(3\)**](#)

Referenced By

[**strongswan_atoul\(3\)**](#), [**strongswan_ttoul\(3\)**](#), [**strtonum\(3\)**](#), [**strtoull\(3\)**](#)



execl(3) - Linux man page

Name

execl, execlp, execl, execv, execvp, execvpe - execute a file

Synopsis

#include <[unistd.h](#)>

extern char **environ;

```
int execl(const char *path, const char *arg, ...);
int execlp(const char *file, const char *arg, ...);
int execl(const char *path, const char *arg,
..., char * const envp[]);
int execv(const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[],
char *const envp[]);
```

Feature Test Macro Requirements for glibc (see [feature test macros](#)(7)):

execvpe(): `_GNU_SOURCE`



Description

The **exec()** family of functions replaces the current process image with a new process image. The functions described in this manual page are front-ends for [execve](#)(2). (See the manual page for [execve](#)(2) for further details about the replacement of the current process image.)

The initial argument for these functions is the name of a file that is to be executed.

The *const char *arg* and subsequent ellipses in the **execl()**, **execlp()**, and **execl()** functions can be thought of as *arg0*, *arg1*, ..., *argn*. Together they describe a list of one or more pointers to null-terminated strings that represent the argument list available to the executed program. The first argument, by convention, should point to the filename associated with the file being executed. The list of arguments *must* be terminated by a NULL pointer, and, since these are variadic functions, this pointer must be cast (*char **) **NULL**.

The **execv()**, **execvp()**, and **execvpe()** functions provide an array of pointers to null-terminated strings that represent the argument list available to the new program. The first argument, by convention, should point to the filename associated with the file being executed. The array of pointers *must* be terminated by a NULL pointer.

The **execle()** and **execvpe()** functions allow the caller to specify the environment of the executed program via the argument *envp*. The *envp* argument is an array of pointers to null-terminated strings and *must* be terminated by a NULL pointer. The other functions take the environment for the new process image from the external variable *environ* in the calling process.

Special semantics for execlp() and execvp()

The **execlp()**, **execvp()**, and **execvpe()** functions duplicate the actions of the shell in searching for an executable file if the specified filename does not contain a slash (/) character. The file is sought in the colon-separated list of directory pathnames specified in the **PATH** environment variable. If this variable isn't defined, the path list defaults to the current directory followed by the list of directories returned by *confstr(_CS_PATH)*. (This **confstr**(3) call typically returns the value `"/bin:/usr/bin"`.)

If the specified filename includes a slash character, then **PATH** is ignored, and the file at the specified pathname is executed.

In addition, certain errors are treated specially.

If permission is denied for a file (the attempted **execve**(2) failed with the error **EACCES**), these functions will continue searching the rest of the search path. If no other file is found, however, they will return with *errno* set to **EACCES**.

If the header of a file isn't recognized (the attempted **execve**(2) failed with the error **ENOEXEC**), these functions will execute the shell (*/bin/sh*) with the path of the file as its first argument. (If this attempt fails, no further searching is done.)



Return Value

The **exec()** functions only return if an error has occurred. The return value is -1, and *errno* is set to indicate the error.

Errors

All of these functions may fail and set *errno* for any of the errors specified for **execve**(2).

Versions

The **execvpe()** function first appeared in glibc 2.11.

Conforming To

POSIX.1-2001, POSIX.1-2008.

The **execvpe()** function is a GNU extension.

Notes

On some other systems, the default path (used when the environment does not contain the variable **PATH**) has the current working directory listed after */bin* and */usr/bin*, as an anti-Trojan-horse measure. Linux uses here the traditional "current directory first" default path.

The behavior of **execlp()** and **execvp()** when errors occur while attempting to execute the file is historic practice, but has not traditionally been documented and is not specified by the POSIX standard. BSD (and possibly other systems) do an automatic sleep and retry if **ETXTBSY** is encountered. Linux treats it as a hard error and returns immediately.

Traditionally, the functions **execlp()** and **execvp()** ignored all errors except for the ones described above and **ENOMEM** and **E2BIG**, upon which they returned. They now return if any error other than the ones described above occurs.

See Also

[sh](#)(1), [execve](#)(2), [fork](#)(2), [ptrace](#)(2), [fexecve](#)(3), [environ](#)(7)

Referenced By

[pipethrough](#)(3), [vacall](#)(3)



fclose(3) - Linux man page

Name

fclose - close a stream

Synopsis

#include <[stdio.h](#)>

int fclose(**FILE** **fp*);

Description

The **fclose()** function flushes the stream pointed to by *fp* (writing any buffered output data using [fflush\(3\)](#)) and closes the underlying file descriptor.

Return Value

Upon successful completion 0 is returned. Otherwise, **EOF** is returned and *errno* is set to indicate the error. In either case any further access (including another call to **fclose()**) to the stream results in undefined behavior.

Errors

EBADF

The file descriptor underlying *fp* is not valid.

The **fclose()** function may also fail and set *errno* for any of the errors specified for the routines [close\(2\)](#), [write\(2\)](#) or [fflush\(3\)](#).

Conforming to

C89, C99.

Notes

Note that **fclose()** only flushes the user-space buffers provided by the C library. To ensure that the data is physically stored on disk the kernel buffers must be flushed too, for example, with [sync\(2\)](#) or [fsync\(2\)](#).

See Also

[close\(2\)](#), [fcloseall\(3\)](#), [fflush\(3\)](#), [fopen\(3\)](#), [setbuf\(3\)](#)

Referenced By



awk(1), **explain**(1), **explain_fclose**(3), **explain_fclose_or_die**(3), **fmemopen**(3),
fopencookie(3), **popen**(3), **xdr**(3)



fopen(3) - Linux man page



Name

fopen, fdopen, freopen - stream open functions

Synopsis

```
#include <stdio.h>
```

```
FILE *fopen(const char *path, const char *mode);
```

```
FILE *fdopen(int fd, const char *mode);
```

```
FILE *freopen(const char *path, const char *mode, FILE *stream);
```

Feature Test Macro Requirements for glibc (see [feature_test_macros\(7\)](#)):

```
fdopen(): _POSIX_C_SOURCE >= 1 || _XOPEN_SOURCE || _POSIX_SOURCE
```

Description

The **fopen()** function opens the file whose name is the string pointed to by *path* and associates a stream with it.

The argument *mode* points to a string beginning with one of the following sequences (possibly followed by additional characters, as described below):

r

Open text file for reading. The stream is positioned at the beginning of the file.

r+

Open for reading and writing. The stream is positioned at the beginning of the file.

w

Truncate file to zero length or create text file for writing. The stream is positioned at the beginning of the file.

w+

Open for reading and writing. The file is created if it does not exist, otherwise it is truncated. The stream is positioned at the beginning of the file.

a

Open for appending (writing at end of file). The file is created if it does not exist. The stream is positioned at the end of the file.



a+

Open for reading and appending (writing at end of file). The file is created if it does not exist. The initial file position for reading is at the beginning of the file, but output is always appended to the end of the file.

The *mode* string can also include the letter 'b' either as a last character or as a character between the characters in any of the two-character strings described above. This is strictly for compatibility with C89 and has no effect; the 'b' is ignored on all POSIX conforming systems, including Linux. (Other systems may treat text files and binary files differently, and adding the 'b' may be a good idea if you do I/O to a binary file and expect that your program may be ported to non-UNIX environments.)

See NOTES below for details of glibc extensions for *mode*.

Any created files will have mode **S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP | S_IROTH | S_IWOTH** (0666), as modified by the process's umask value (see [umask\(2\)](#)).

Reads and writes may be intermixed on read/write streams in any order. Note that ANSI C requires that a file positioning function intervene between output and input, unless an input operation encounters end-of-file. (If this condition is not met, then a read is allowed to return the result of writes other than the most recent.) Therefore it is good practice (and indeed sometimes necessary under Linux) to put an [fseek\(3\)](#) or [fgetpos\(3\)](#) operation between write and read operations on such a stream. This operation may be an apparent no-op (as in `fseek(..., 0L, SEEK_CUR)`) called for its synchronizing side effect.

Opening a file in append mode (**a** as the first character of *mode*) causes all subsequent write operations to this stream to occur at end-of-file, as if preceded the call:

```
fseek(stream, 0, SEEK_END);
```

The **fdopen()** function associates a stream with the existing file descriptor, *fd*. The *mode* of the stream (one of the values "r", "r+", "w", "w+", "a", "a+") must be compatible with the mode of the file descriptor. The file position indicator of the new stream is set to that belonging to *fd*, and the error and end-of-file indicators are cleared. Modes "w" or "w+" do not cause truncation of the file. The file descriptor is not dup'ed, and will be closed when the stream created by **fdopen()** is closed. The result of applying **fdopen()** to a shared memory object is undefined.

The **freopen()** function opens the file whose name is the string pointed to by *path* and associates the stream pointed to by *stream* with it. The original stream (if it exists) is closed. The *mode* argument is used just as in the **fopen()** function. The primary use of the **freopen()** function is to change the file associated with a standard text stream (*stderr*, *stdin*, or *stdout*).



Return Value

Upon successful completion **fopen()**, **fdopen()** and **freopen()** return a *FILE* pointer. Otherwise, NULL is returned and *errno* is set to indicate the error.

Errors

EINVAL

The *mode* provided to **fopen()**, **fdopen()**, or **freopen()** was invalid.

The **fopen()**, **fdopen()** and **freopen()** functions may also fail and set *errno* for any of the errors specified for the routine **malloc**(3).

The **fopen()** function may also fail and set *errno* for any of the errors specified for the routine **open**(2).

The **fdopen()** function may also fail and set *errno* for any of the errors specified for the routine **fcntl**(2).

The **freopen()** function may also fail and set *errno* for any of the errors specified for the routines **open**(2), **fclose**(3) and **fflush**(3).

Conforming To

The **fopen()** and **freopen()** functions conform to C89. The **fdopen()** function conforms to POSIX.1-1990.

Notes

Glibc notes

The GNU C library allows the following extensions for the string specified in *mode*:

c (since glibc 2.3.3)

Do not make the open operation, or subsequent read and write operations, thread cancellation points. This flag is ignored for **fdopen()**.

e (since glibc 2.7)

Open the file with the **O_CLOEXEC** flag. See **open**(2) for more information. This flag is ignored for **fdopen()**.

m (since glibc 2.3)

Attempt to access the file using **mmap**(2), rather than I/O system calls (**read**(2), **write**(2)). Currently, use of **mmap**(2) is only attempted for a file opened for reading.

x

Open the file exclusively (like the **O_EXCL** flag of **open**(2)). If the file already exists, **fopen()** fails, and sets *errno* to **EEXIST**. This flag is ignored for **fdopen()**.

In addition to the above characters, **fopen()** and **freopen()** support the following syntax *mode*:



,ccs=string

The given *string* is taken as the name of a coded character set and the stream is marked as wide-oriented. Thereafter, internal conversion functions convert I/O to and from the character set *string*. If the **,ccs=string** syntax is not specified, then the wide-orientation of the stream is determined by the first file operation. If that operation is a wide-character operation, the stream is marked wide-oriented, and functions to convert to the coded character set are loaded.

Bugs

When parsing for individual flag characters in *mode* (i.e., the characters preceding the "ccs" specification), the glibc implementation of **fopen()** and **freopen()** limits the number of characters examined in *mode* to 7 (or, in glibc versions before 2.14, to 6, which was not enough to include possible specifications such as "rb+cmxe"). The current implementation of **fdopen()** parses at most 5 characters in *mode*.

See Also

[**open\(2\)**](#), [**fclose\(3\)**](#), [**fileno\(3\)**](#), [**fmemopen\(3\)**](#), [**fopencookie\(3\)**](#)

Referenced By

[**explain\(1\)**](#), [**explain\(3\)**](#), [**explain_fopen\(3\)**](#), [**explain_fopen_or_die\(3\)**](#), [**fcloseall\(3\)**](#), [**fgetgrent\(3\)**](#), [**fgetln\(3\)**](#), [**fgetpwent\(3\)**](#), [**fgets\(3\)**](#), [**fgetwc\(3\)**](#), [**fgetws\(3\)**](#), [**fputwc\(3\)**](#), [**fputws\(3\)**](#), [**getline\(3\)**](#), [**getmntent\(3\)**](#), [**gvpr\(1\)**](#), [**mbkfopen\(3\)**](#), [**openfilechnl\(3\)**](#), [**perlfunc\(1\)**](#), [**perlopentut\(1\)**](#), [**popen\(3\)**](#), [**pthread_getattr_np\(3\)**](#), [**putc\(3\)**](#), [**rfio_fopen\(3\)**](#), [**setbuf\(3\)**](#), [**snmpd.conf\(5\)**](#), [**stdin\(3\)**](#), [**tcl_openfilechannel\(3\)**](#), [**tstdio\(3\)**](#), [**xs\(1\)**](#), [**zzip_freopen\(3\)**](#)

getline(3) - Linux man page

Name

getline, getdelim - delimited string input

Synopsis

```
#include <stdio.h>
```

```
ssize_t getline(char **lineptr, size_t *n, FILE *stream);
```

```
ssize_t getdelim(char **lineptr, size_t *n, int delim, FILE *stream);
```

Feature Test Macro Requirements for glibc (see [feature_test_macros\(7\)](#)):

getline(), **getdelim()**:

Since glibc 2.10:

`_POSIX_C_SOURCE >= 200809L` || `_XOPEN_SOURCE >= 700`

Before glibc 2.10:

`_GNU_SOURCE`

Description



getline() reads an entire line from *stream*, storing the address of the buffer containing the text into **lineptr*. The buffer is null-terminated and includes the newline character, if one was found.

If **lineptr* is NULL, then **getline()** will allocate a buffer for storing the line, which should be freed by the user program. (In this case, the value in **n* is ignored.)

Alternatively, before calling **getline()**, **lineptr* can contain a pointer to a [malloc\(3\)](#)-allocated buffer **n* bytes in size. If the buffer is not large enough to hold the line, **getline()** resizes it with [realloc\(3\)](#), updating **lineptr* and **n* as necessary.

In either case, on a successful call, **lineptr* and **n* will be updated to reflect the buffer address and allocated size respectively.

getdelim() works like **getline()**, except that a line delimiter other than newline can be specified as the *delimiter* argument. As with **getline()**, a delimiter character is not added if one was not present in the input before end of file was reached.

Return Value

On success, **getline()** and **getdelim()** return the number of characters read, including the delimiter character, but not including the terminating null byte. This value can be used to

handle embedded null bytes in the line read.

Both functions return -1 on failure to read a line (including end-of-file condition).

Errors

EINVAL

Bad arguments (*n* or *lineptr* is NULL, or *stream* is not valid).

Versions

These functions are available since libc 4.6.27.

Conforming To

Both **getline()** and **getdelim()** were originally GNU extensions. They were standardized in POSIX.1-2008.

Example

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>

int
main(void)
{
    FILE *fp;
    char *line = NULL;
    size_t len = 0;
    ssize_t read;

    fp = fopen("/etc/motd", "r");
    if (fp == NULL)
        exit(EXIT_FAILURE);

    while ((read = getline(&line, &len, fp)) != -1) {
        printf("Retrieved line of length %zu :\n", read);
        printf("%s", line);
    }

    free(line);
    exit(EXIT_SUCCESS);
}
```



See Also

[read](#)(2), [fgets](#)(3), [fopen](#)(3), [fread](#)(3), [gets](#)(3), [scanf](#)(3)

Referenced By

[rpmatch](#)(3)



isspace(3) - Linux man page

Name

isalnum, isalpha, isascii, isblank, iscntrl, isdigit, isgraph, islower, isprint, ispunct, isspace, isupper, isxdigit - character classification routines

Synopsis

```
#include <ctype.h>
```

```
int isalnum(int c);
int isalpha(int c);
int isascii(int c);
int isblank(int c);
int iscntrl(int c);
int isdigit(int c);
int isgraph(int c);
int islower(int c);
int isprint(int c);
int ispunct(int c);
int isspace(int c);
int isupper(int c);
int isxdigit(int c);
```



Feature Test Macro Requirements for glibc (see [feature test macros\(7\)](#)):

isascii():

`_BSD_SOURCE || _SVID_SOURCE || _XOPEN_SOURCE`

isblank():

`_XOPEN_SOURCE >= 600 || _ISOC99_SOURCE || _POSIX_C_SOURCE >= 200112L;`
or `cc -std=c99`

Description

These functions check whether *c*, which must have the value of an *unsigned char* or **EOF**, falls into a certain character class according to the current locale.

isalnum()

checks for an alphanumeric character; it is equivalent to **(isalpha(c) || isdigit(c))**.

isalpha()

checks for an alphabetic character; in the standard **"C"** locale, it is equivalent to **(isupper(c) || islower(c))**. In some locales, there may be additional characters for which **isalpha()** is true—letters which are neither upper case nor lower case.

isascii()

checks whether *c* is a 7-bit *unsigned char* value that fits into the ASCII character set.

isblank()

checks for a blank character; that is, a space or a tab.

isctrl()

checks for a control character.

isdigit()

checks for a digit (0 through 9).

isgraph()

checks for any printable character except space.

islower()

checks for a lower-case character.

isprint()

checks for any printable character including space.

ispunct()

checks for any printable character which is not a space or an alphanumeric character.

isspace()

checks for white-space characters. In the "**C**" and "**POSIX**" locales, these are: space, form-feed (`'\f'`), newline (`'\n'`), carriage return (`'\r'`), horizontal tab (`'\t'`), and vertical tab (`'\v'`).

isupper()

checks for an uppercase letter.

isxdigit()

checks for a hexadecimal digits, that is, one of

0 1 2 3 4 5 6 7 8 9 a b c d e f A B C D E F.



Return Value

The values returned are nonzero if the character *c* falls into the tested class, and a zero value if not.

Conforming To

C99, 4.3BSD. C89 specifies all of these functions except **isascii()** and **isblank()**. **isascii()** is a BSD extension and is also an SVr4 extension. **isblank()** conforms to POSIX.1-2001 and C99 7.4.1.3. POSIX.1-2008 marks **isascii()** as obsolete, noting that it cannot be used portably in a localized application.

Notes

The details of what characters belong into which class depend on the current locale. For example, **isupper()** will not recognize an A-umlaut (Ä,,) as an uppercase letter in the default **C** locale.

See Also

iswalnum(3), **iswalpha**(3), **iswblank**(3), **iswcntrl**(3), **iswdigit**(3), **iswgraph**(3),
iswlower(3), **iswprint**(3), **iswpunct**(3), **iswspace**(3), **iswupper**(3), **iswxdigit**(3),
setlocale(3), **toascii**(3), **tolower**(3), **toupper**(3), **ascii**(7), **locale**(7)

Referenced By

scanf(3), **strtod**(3), **strtol**(3), **strtonum**(3), **strtoull**(3), **wctype**(3)



malloc(3) - Linux man page



Name

malloc, free, calloc, realloc - allocate and free dynamic memory

Synopsis

```
#include <stdlib.h>
```

```
void *malloc(size_t size);  
void free(void *ptr);  
void *calloc(size_t nmemb, size_t size);  
void *realloc(void *ptr, size_t size);
```

Description

The **malloc()** function allocates *size* bytes and returns a pointer to the allocated memory. *The memory is not initialized.* If *size* is 0, then **malloc()** returns either NULL, or a unique pointer value that can later be successfully passed to **free()**.

The **free()** function frees the memory space pointed to by *ptr*, which must have been returned by a previous call to **malloc()**, **calloc()** or **realloc()**. Otherwise, or if *free(ptr)* has already been called before, undefined behavior occurs. If *ptr* is NULL, no operation is performed.

The **calloc()** function allocates memory for an array of *nmemb* elements of *size* bytes each and returns a pointer to the allocated memory. The memory is set to zero. If *nmemb* or *size* is 0, then **calloc()** returns either NULL, or a unique pointer value that can later be successfully passed to **free()**.

The **realloc()** function changes the size of the memory block pointed to by *ptr* to *size* bytes. The contents will be unchanged in the range from the start of the region up to the minimum of the old and new sizes. If the new size is larger than the old size, the added memory will *not* be initialized. If *ptr* is NULL, then the call is equivalent to *malloc(size)*, for all values of *size*; if *size* is equal to zero, and *ptr* is not NULL, then the call is equivalent to *free(ptr)*. Unless *ptr* is NULL, it must have been returned by an earlier call to **malloc()**, **calloc()** or **realloc()**. If the area pointed to was moved, a *free(ptr)* is done.

Return Value

The **malloc()** and **calloc()** functions return a pointer to the allocated memory that is suitably aligned for any kind of variable. On error, these functions return NULL. NULL may also be returned by a successful call to **malloc()** with a *size* of zero, or by a successful call to **calloc()** with *nmemb* or *size* equal to zero.



The **free()** function returns no value.

The **realloc()** function returns a pointer to the newly allocated memory, which is suitably aligned for any kind of variable and may be different from *ptr*, or NULL if the request fails. If *size* was equal to 0, either NULL or a pointer suitable to be passed to **free()** is returned. If **realloc()** fails the original block is left untouched; it is not freed or moved.

Conforming To

C89, C99.

Notes

By default, Linux follows an optimistic memory allocation strategy. This means that when **malloc()** returns non-NULL there is no guarantee that the memory really is available. In case it turns out that the system is out of memory, one or more processes will be killed by the OOM killer. For more information, see the description of */proc/sys/vm/overcommit_memory* and */proc/sys/vm/oom_adj* in **proc(5)**, and the Linux kernel source file *Documentation/vm/overcommit-accounting*.

Normally, **malloc()** allocates memory from the heap, and adjusts the size of the heap as required, using **sbrk(2)**. When allocating blocks of memory larger than **MMAP_THRESHOLD** bytes, the glibc **malloc()** implementation allocates the memory as a private anonymous mapping using **mmap(2)**. **MMAP_THRESHOLD** is 128 kB by default, but is adjustable using **mallopt(3)**. Allocations performed using **mmap(2)** are unaffected by the **RLIMIT_DATA** resource limit (see **getrlimit(2)**).

To avoid corruption in multithreaded applications, mutexes are used internally to protect the memory-management data structures employed by these functions. In a multithreaded application in which threads simultaneously allocate and free memory, there could be contention for these mutexes. To scalably handle memory allocation in multithreaded applications, glibc creates additional *memory allocation arenas* if mutex contention is detected. Each arena is a large region of memory that is internally allocated by the system (using **brk(2)** or **mmap(2)**), and managed with its own mutexes.

The UNIX 98 standard requires **malloc()**, **calloc()**, and **realloc()** to set *errno* to **ENOMEM** upon failure. Glibc assumes that this is done (and the glibc versions of these routines do this); if you use a private malloc implementation that does not set *errno*, then certain library routines may fail without having a reason in *errno*.

Crashes in **malloc()**, **calloc()**, **realloc()**, or **free()** are almost always related to heap corruption, such as overflowing an allocated chunk or freeing the same pointer twice.

Recent versions of Linux libc (later than 5.4.23) and glibc (2.x) include a **malloc()** implementation which is tunable via environment variables. For details, see **mallopt(3)**.

See Also



[brk\(2\)](#), [mmap\(2\)](#), [alloca\(3\)](#), [malloc_get_state\(3\)](#), [malloc_info\(3\)](#), [malloc_trim\(3\)](#), [malloc_usable_size\(3\)](#), [mallopt\(3\)](#), [mcheck\(3\)](#), [mtrace\(3\)](#), [posix_memalign\(3\)](#)

Referenced By

[all_hash\(3\)](#), [all_list\(3\)](#), [all_queue\(3\)](#), [argz_add\(3\)](#), [asprintf\(3\)](#), [avc_init\(3\)](#), [backtrace\(3\)](#), [cdecl\(1\)](#), [cdt\(3\)](#), [cfree\(3\)](#), [cp_string_create\(3\)](#), [cp_string_cstrdup\(3\)](#), [cp_string_dup\(3\)](#), [cpu_set\(3\)](#), [csch\(1\)](#), [dbopen\(3\)](#), [diff\(3\)](#), [duma\(3\)](#), [end\(3\)](#), [environ\(5\)](#), [environ\(7\)](#), [explain\(1\)](#), [explain\(3\)](#), [explain_malloc\(3\)](#), [explain_malloc_or_die\(3\)](#), [explain_output\(3\)](#), [fgetln\(3\)](#), [fio\(1\)](#), [fmemopen\(3\)](#), [fopen\(3\)](#), [fseek\(3\)](#), [fts\(3\)](#), [ftw\(3\)](#), [gc\(3\)](#), [genders_index_attrvals\(3\)](#), [genders_load_data\(3\)](#), [genders_nodelist_create\(3\)](#), [genders_query\(3\)](#), [genders_testquery\(3\)](#), [getcwd\(2\)](#), [getifaddrs\(3\)](#), [getline\(3\)](#), [glob\(3\)](#), [hsearch\(3\)](#), [if_nameindex\(3\)](#), [im_add_eval_callback\(3\)](#), [im_region_local\(3\)](#), [lber-memory\(3\)](#), [ldap_err2string\(3\)](#), [ldap_memfree\(3\)](#), [libefence\(3\)](#), [lwres_context\(3\)](#), [mallinfo\(3\)](#), [malloc_hook\(3\)](#), [malloc_stats\(3\)](#), [memtester\(8\)](#), [mhash\(3\)](#), [mlockall\(2\)](#), [mpool\(3\)](#), [mremap\(2\)](#), [nodeupdown_get_up_nodes_list\(3\)](#), [nodeupdown_get_up_nodes_string\(3\)](#), [nodeupdown_is_node_up\(3\)](#), [nodeupdown_load_data\(3\)](#), [nodeupdown_nodelist_create\(3\)](#), [numa\(3\)](#), [perldebguts\(1\)](#), [perlipc\(1\)](#), [pmdafetch\(3\)](#), [pmdainstance\(3\)](#), [pmdatext\(3\)](#), [pmfreeresult\(3\)](#), [pth\(3\)](#), [pthread_setcanceltype\(3\)](#), [radixsort\(3\)](#), [readline\(3\)](#), [reallocf\(3\)](#), [realpath\(3\)](#), [samefile\(1\)](#), [scandir\(3\)](#), [setbuf\(3\)](#), [setmode\(3\)](#), [sphere_i\(3\)](#), [strndup\(3\)](#), [talloc\(3\)](#), [tempnam\(3\)](#), [tst\(3\)](#), [varnishd\(1\)](#), [wcsdup\(3\)](#), [wnsearch\(3\)](#), [zzip_disk_open\(3\)](#)

sprintf(3) - Linux man page



Name

printf, fprintf, sprintf, snprintf, vprintf, vfprintf, vsprintf, vsnprintf - formatted output conversion

Synopsis

#include <[stdio.h](#)>

```
int printf(const char *format, ...);  
int fprintf(FILE *stream, const char *format, ...);  
int sprintf(char *str, const char *format, ...);  
int snprintf(char *str, size_t size, const char *format, ...);
```

#include <[stdarg.h](#)>

```
int vprintf(const char *format, va_list ap);  
int vfprintf(FILE *stream, const char *format, va_list ap);  
int vsprintf(char *str, const char *format, va_list ap);  
int vsnprintf(char *str, size_t size, const char *format, va_list ap);
```

Feature Test Macro Requirements for glibc (see [feature test macros](#)(7)):

```
snprintf(), vsnprintf():  
    _BSD_SOURCE || _XOPEN_SOURCE >= 500 || _ISOC99_SOURCE ||  
    _POSIX_C_SOURCE >= 200112L;  
    or cc -std=c99
```

Description

The functions in the **printf()** family produce output according to a *format* as described below. The functions **printf()** and **vprintf()** write output to *stdout*, the standard output stream; **fprintf()** and **vfprintf()** write output to the given output *stream*; **sprintf()**, **snprintf()**, **vsprintf()** and **vsnprintf()** write to the character string *str*.

The functions **snprintf()** and **vsnprintf()** write at most *size* bytes (including the terminating null byte ('\0')) to *str*.

The functions **vprintf()**, **vfprintf()**, **vsprintf()**, **vsnprintf()** are equivalent to the functions **printf()**, **fprintf()**, **sprintf()**, **snprintf()**, respectively, except that they are called with a *va_list* instead of a variable number of arguments. These functions do not call the *va_end* macro. Because they invoke the *va_arg* macro, the value of *ap* is undefined after the call. See [stdarg\(3\)](#).



These eight functions write the output under the control of a *format* string that specifies how subsequent arguments (or arguments accessed via the variable-length argument facilities of **stdarg(3)**) are converted for output.

C99 and POSIX.1-2001 specify that the results are undefined if a call to **sprintf()**, **snprintf()**, **vsprintf()**, or **vsnprintf()** would cause copying to take place between objects that overlap (e.g., if the target string array and one of the supplied input arguments refer to the same buffer). See NOTES.

Return value

Upon successful return, these functions return the number of characters printed (excluding the null byte used to end output to strings).

The functions **snprintf()** and **vsnprintf()** do not write more than *size* bytes (including the terminating null byte ('\0')). If the output was truncated due to this limit then the return value is the number of characters (excluding the terminating null byte) which would have been written to the final string if enough space had been available. Thus, a return value of *size* or more means that the output was truncated. (See also below under NOTES.)

If an output error is encountered, a negative value is returned.

Format of the format string

The format string is a character string, beginning and ending in its initial shift state, if any. The format string is composed of zero or more directives: ordinary characters (not **%**), which are copied unchanged to the output stream; and conversion specifications, each of which results in fetching zero or more subsequent arguments. Each conversion specification is introduced by the character **%**, and ends with a *conversion specifier*. In between there may be (in this order) zero or more *flags*, an optional minimum *field width*, an optional *precision* and an optional *length modifier*.

The arguments must correspond properly (after type promotion) with the conversion specifier. By default, the arguments are used in the order given, where each ***** and each conversion specifier asks for the next argument (and it is an error if insufficiently many arguments are given). One can also specify explicitly which argument is taken, at each place where an argument is required, by writing **"%m\$"** instead of **"%"** and **"*m\$"** instead of *****, where the decimal integer *m* denotes the position in the argument list of the desired argument, indexed starting from 1. Thus,

```
printf("%*d", width, num);
```

and

```
printf("%2$*1$d", width, num);
```



are equivalent. The second style allows repeated references to the same argument. The C99 standard does not include the style using '\$', which comes from the Single UNIX Specification. If the style using '\$' is used, it must be used throughout for all conversions taking an argument and all width and precision arguments, but it may be mixed with "%%" formats which do not consume an argument. There may be no gaps in the numbers of arguments specified using '\$'; for example, if arguments 1 and 3 are specified, argument 2 must also be specified somewhere in the format string.

For some numeric conversions a radix character ("decimal point") or thousands' grouping character is used. The actual character used depends on the **LC_NUMERIC** part of the locale. The POSIX locale uses '.' as radix character, and does not have a grouping character. Thus,

```
printf("%.2f", 1234567.89);
```

results in "1234567.89" in the POSIX locale, in "1234567,89" in the nl_NL locale, and in "1.234.567,89" in the da_DK locale.

The flag characters

The character % is followed by zero or more of the following flags:

#

The value should be converted to an "alternate form". For **o** conversions, the first character of the output string is made zero (by prefixing a 0 if it was not zero already). For **x** and **X** conversions, a nonzero result has the string "0x" (or "0X" for **X** conversions) prepended to it. For **a**, **A**, **e**, **E**, **f**, **F**, **g**, and **G** conversions, the result will always contain a decimal point, even if no digits follow it (normally, a decimal point appears in the results of those conversions only if a digit follows). For **g** and **G** conversions, trailing zeros are not removed from the result as they would otherwise be. For other conversions, the result is undefined.

0

The value should be zero padded. For **d**, **i**, **o**, **u**, **x**, **X**, **a**, **A**, **e**, **E**, **f**, **F**, **g**, and **G** conversions, the converted value is padded on the left with zeros rather than blanks. If the **0** and **-** flags both appear, the **0** flag is ignored. If a precision is given with a numeric conversion (**d**, **i**, **o**, **u**, **x**, and **X**), the **0** flag is ignored. For other conversions, the behavior is undefined.

-

The converted value is to be left adjusted on the field boundary. (The default is right justification.) Except for **n** conversions, the converted value is padded on the right with blanks, rather than on the left with blanks or zeros. A **-** overrides a **0** if both are given.

.

(a space) A blank should be left before a positive number (or empty string) produced by a signed conversion.



+

A sign (+ or -) should always be placed before a number produced by a signed conversion. By default a sign is used only for negative numbers. A **+** overrides a space if both are used.

The five flag characters above are defined in the C standard. The SUSv2 specifies one further flag character.

,

For decimal conversion (**i, d, u, f, F, g, G**) the output is to be grouped with thousands' grouping characters if the locale information indicates any. Note that many versions of **gcc**(1) cannot parse this option and will issue a warning. SUSv2 does not include **%F**.

glibc 2.2 adds one further flag character.

I

For decimal integer conversion (**i, d, u**) the output uses the locale's alternative output digits, if any. For example, since glibc 2.2.3 this will give Arabic-Indic digits in the Persian ("fa_IR") locale.

The field width

An optional decimal digit string (with nonzero first digit) specifying a minimum field width. If the converted value has fewer characters than the field width, it will be padded with spaces on the left (or right, if the left-adjustment flag has been given). Instead of a decimal digit string one may write **"*"** or **"*m\$"** (for some decimal integer *m*) to specify that the field width is given in the next argument, or in the *m*-th argument, respectively, which must be of type *int*. A negative field width is taken as a **'-'** flag followed by a positive field width. In no case does a nonexistent or small field width cause truncation of a field; if the result of a conversion is wider than the field width, the field is expanded to contain the conversion result.

The precision

An optional precision, in the form of a period (**'.'**) followed by an optional decimal digit string. Instead of a decimal digit string one may write **"*"** or **"*m\$"** (for some decimal integer *m*) to specify that the precision is given in the next argument, or in the *m*-th argument, respectively, which must be of type *int*. If the precision is given as just **'.'**, or the precision is negative, the precision is taken to be zero. This gives the minimum number of digits to appear for **d, i, o, u, x,** and **X** conversions, the number of digits to appear after the radix character for **a, A, e, E, f,** and **F** conversions, the maximum number of significant digits for **g** and **G** conversions, or the maximum number of characters to be printed from a string for **s** and **S** conversions.



The length modifier

Here, "integer conversion" stands for **d**, **i**, **o**, **u**, **x**, or **X** conversion.

hh

A following integer conversion corresponds to a *signed char* or *unsigned char* argument, or a following **n** conversion corresponds to a pointer to a *signed char* argument.

h

A following integer conversion corresponds to a *short int* or *unsigned short int* argument, or a following **n** conversion corresponds to a pointer to a *short int* argument.

l

(ell) A following integer conversion corresponds to a *long int* or *unsigned long int* argument, or a following **n** conversion corresponds to a pointer to a *long int* argument, or a following **c** conversion corresponds to a *wint_t* argument, or a following **s** conversion corresponds to a pointer to *wchar_t* argument.

ll

(ell-ell). A following integer conversion corresponds to a *long long int* or *unsigned long long int* argument, or a following **n** conversion corresponds to a pointer to a *long long int* argument.

L

A following **a**, **A**, **e**, **E**, **f**, **F**, **g**, or **G** conversion corresponds to a *long double* argument. (C99 allows %LF, but SUSv2 does not.)

q

("quad". 4.4BSD and Linux libc5 only. Don't use.) This is a synonym for **ll**.

j

A following integer conversion corresponds to an *intmax_t* or *uintmax_t* argument.

z

A following integer conversion corresponds to a *size_t* or *ssize_t* argument. (Linux libc5 has **Z** with this meaning. Don't use it.)

t

A following integer conversion corresponds to a *ptrdiff_t* argument.

The SUSv2 only knows about the length modifiers **h** (in **hd, hi, ho, hx, hX, hn**) and **l** (**ld, li, lo, lx, lX, ln, lc, ls**) and **L** (in **Le, LE, Lf, Lg, LG**).



The conversion specifier

A character that specifies the type of conversion to be applied. The conversion specifiers and their meanings are:

d, i

The *int* argument is converted to signed decimal notation. The precision, if any, gives the minimum number of digits that must appear; if the converted value requires fewer digits, it is padded on the left with zeros. The default precision is 1. When 0 is printed with an explicit precision 0, the output is empty.

o, u, x, X

The *unsigned int* argument is converted to unsigned octal (**o**), unsigned decimal (**u**), or unsigned hexadecimal (**x** and **X**) notation. The letters **abcdef** are used for **x** conversions; the letters **ABCDEF** are used for **X** conversions. The precision, if any, gives the minimum number of digits that must appear; if the converted value requires fewer digits, it is padded on the left with zeros. The default precision is 1. When 0 is printed with an explicit precision 0, the output is empty.

e, E

The *double* argument is rounded and converted in the style `[-]d.ddde $\pm$ dd` where there is one digit before the decimal-point character and the number of digits after it is equal to the precision; if the precision is missing, it is taken as 6; if the precision is zero, no decimal-point character appears. An **E** conversion uses the letter **E** (rather than **e**) to introduce the exponent. The exponent always contains at least two digits; if the value is zero, the exponent is 00.

f, F

The *double* argument is rounded and converted to decimal notation in the style `[-]ddd.ddd`, where the number of digits after the decimal-point character is equal to the precision specification. If the precision is missing, it is taken as 6; if the precision is explicitly zero, no decimal-point character appears. If a decimal point appears, at least one digit appears before it.

(The SUSv2 does not know about **F** and says that character string representations for infinity and NaN may be made available. The C99 standard specifies `"[-]inf"` or `"[-]infinity"` for infinity, and a string starting with `"nan"` for NaN, in the case of **f** conversion, and `"[-]INF"` or `"[-]INFINITY"` or `"NAN*"` in the case of **F** conversion.)

g, G

The *double* argument is converted in style **f** or **e** (or **F** or **E** for **G** conversions). The precision specifies the number of significant digits. If the precision is missing, 6 digits are

given; if the precision is zero, it is treated as 1. Style **e** is used if the exponent from it conversion is less than -4 or greater than or equal to the precision. Trailing zeros are removed from the fractional part of the result; a decimal point appears only if it is followed by at least one digit.



a, A

(C99; not in SUSv2) For **a** conversion, the *double* argument is converted to hexadecimal notation (using the letters abcdef) in the style `[-]0xh.hhhhp±`; for **A** conversion the prefix **0X**, the letters ABCDEF, and the exponent separator **P** is used. There is one hexadecimal digit before the decimal point, and the number of digits after it is equal to the precision. The default precision suffices for an exact representation of the value if an exact representation in base 2 exists and otherwise is sufficiently large to distinguish values of type *double*. The digit before the decimal point is unspecified for nonnormalized numbers, and nonzero but otherwise unspecified for normalized numbers.

c

If no **l** modifier is present, the *int* argument is converted to an *unsigned char*, and the resulting character is written. If an **l** modifier is present, the *wint_t* (wide character) argument is converted to a multibyte sequence by a call to the **wcrtomb**(3) function, with a conversion state starting in the initial state, and the resulting multibyte string is written.

s

If no **l** modifier is present: The *const char ** argument is expected to be a pointer to an array of character type (pointer to a string). Characters from the array are written up to (but not including) a terminating null byte ('\0'); if a precision is specified, no more than the number specified are written. If a precision is given, no null byte need be present; if the precision is not specified, or is greater than the size of the array, the array must contain a terminating null byte.

If an **l** modifier is present: The *const wchar_t ** argument is expected to be a pointer to an array of wide characters. Wide characters from the array are converted to multibyte characters (each by a call to the **wcrtomb**(3) function, with a conversion state starting in the initial state before the first wide character), up to and including a terminating null wide character. The resulting multibyte characters are written up to (but not including) the terminating null byte. If a precision is specified, no more bytes than the number specified are written, but no partial multibyte characters are written. Note that the precision determines the number of *bytes* written, not the number of *wide characters* or *screen positions*. The array must contain a terminating null wide character, unless a precision is given and it is so small that the number of bytes written exceeds it before the end of the array is reached.

C

(Not in C99, but in SUSv2.) Synonym for **lc**. Don't use.



S

(Not in C99, but in SUSv2.) Synonym for **ls**. Don't use.

P

The *void ** pointer argument is printed in hexadecimal (as if by **%#x** or **%#lx**).

n

The number of characters written so far is stored into the integer indicated by the *int ** (or variant) pointer argument. No argument is converted.

m

(Glibc extension.) Print output of *strerror(errno)*. No argument is required.

%

A '%' is written. No argument is converted. The complete conversion specification is '%%'.

Conforming To

The **fprintf()**, **printf()**, **sprintf()**, **vprintf()**, **vfprintf()**, and **vsprintf()** functions conform to C89 and C99. The **snprintf()** and **vsnprintf()** functions conform to C99.

Concerning the return value of **snprintf()**, SUSv2 and C99 contradict each other: when **snprintf()** is called with *size*=0 then SUSv2 stipulates an unspecified return value less than 1, while C99 allows *str* to be NULL in this case, and gives the return value (as always) as the number of characters that would have been written in case the output string has been large enough.

Linux libc4 knows about the five C standard flags. It knows about the length modifiers **h**, **l**, **L**, and the conversions **c**, **d**, **e**, **E**, **f**, **F**, **g**, **G**, **i**, **n**, **o**, **p**, **s**, **u**, **x**, and **X**, where **F** is a synonym for **f**. Additionally, it accepts **D**, **O**, and **U** as synonyms for **ld**, **lo**, and **lu**. (This is bad, and caused serious bugs later, when support for **%D** disappeared.) No locale-dependent radix character, no thousands' separator, no NaN or infinity, no "%m\$" and "*m\$".

Linux libc5 knows about the five C standard flags and the ' flag, locale, "%m\$" and "*m\$". It knows about the length modifiers **h**, **l**, **L**, **Z**, and **q**, but accepts **L** and **q** both for *long double* and for *long long int* (this is a bug). It no longer recognizes **F**, **D**, **O**, and **U**, but adds the conversion character **m**, which outputs *strerror(errno)*.

glibc 2.0 adds conversion characters **C** and **S**.

glibc 2.1 adds length modifiers **hh**, **j**, **t**, and **z** and conversion characters **a** and **A**.

glibc 2.2 adds the conversion character **F** with C99 semantics, and the flag character **F**.



Notes

Some programs imprudently rely on code such as the following

```
sprintf(buf, "%s some further text", buf);
```

to append text to *buf*. However, the standards explicitly note that the results are undefined if source and destination buffers overlap when calling **sprintf()**, **snprintf()**, **vsprintf()**, and **vsnprintf()**. Depending on the version of **gcc**(1) used, and the compiler options employed, calls such as the above will **not** produce the expected results.

The glibc implementation of the functions **snprintf()** and **vsnprintf()** conforms to the C99 standard, that is, behaves as described above, since glibc version 2.1. Until glibc 2.0.6 they would return -1 when the output was truncated.

Bugs

Because **sprintf()** and **vsprintf()** assume an arbitrarily long string, callers must be careful not to overflow the actual space; this is often impossible to assure. Note that the length of the strings produced is locale-dependent and difficult to predict. Use **snprintf()** and **vsnprintf()** instead (or **asprintf(3)** and **vasprintf(3)**).

Linux libc4.[45] does not have a **snprintf()**, but provides a libbsd that contains an **snprintf()** equivalent to **sprintf()**, that is, one that ignores the *size* argument. Thus, the use of **snprintf()** with early libc4 leads to serious security problems.

Code such as **printf(foo)**; often indicates a bug, since *foo* may contain a % character. If *foo* comes from untrusted user input, it may contain **%n**, causing the **printf()** call to write to memory and creating a security hole.

Example

To print *Pi* to five decimal places:

```
#include <math.h>
#include <stdio.h>
fprintf(stdout, "pi = %.5f\n", 4 * atan(1.0));
```

To print a date and time in the form "Sunday, July 3, 10:02", where *weekday* and *month* are pointers to strings:

```
#include <stdio.h>
fprintf(stdout, "%s, %s %d, %.2d:%.2d\n",
        weekday, month, day, hour, min);
```

Many countries use the day-month-year order. Hence, an internationalized version must be able to print the arguments in an order specified by the format:



```
#include <stdio.h>
fprintf(stdout, format,
        weekday, month, day, hour, min);
```

where *format* depends on locale, and may permute the arguments. With the value:

```
"%1$s, %3$d. %2$s, %4$d:%5$.2d\n"
```

one might obtain "Sonntag, 3. Juli, 10:02".

To allocate a sufficiently large string and print into it (code correct for both glibc 2.0 and glibc 2.1):

If truncation occurs in glibc versions prior to 2.0.6, this is treated as an error instead of being handled gracefully.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>

char *
make_message(const char *fmt, ...)
{
    int n;
    int size = 100;    /* Guess we need no more than 100 bytes */
    char *p, *np;
    va_list ap;

    if ((p = malloc(size)) == NULL)
        return NULL;

    while (1) {

        /* Try to print in the allocated space */

        va_start(ap, fmt);
        n = vsnprintf(p, size, fmt, ap);
        va_end(ap);

        /* Check error code */

        if (n < 0)
```



```
        return NULL;

/* If that worked, return the string */

if (n < size)
    return p;

/* Else try again with more space */

size = n + 1;      /* Precisely what is needed */

if ((np = realloc (p, size)) == NULL) {
    free(p);
    return NULL;
} else {
    p = np;
}
}
```

See Also

[**printf**\(1\)](#), [**asprintf**\(3\)](#), [**dprintf**\(3\)](#), [**scanf**\(3\)](#), [**setlocale**\(3\)](#), [**wcrtomb**\(3\)](#), [**wprintf**\(3\)](#), [**locale**\(5\)](#)

Referenced By

[**awk**\(1\)](#), [**batcher**\(8\)](#), [**buffchan**\(8\)](#), [**curl_mprintf**\(3\)](#), [**ecvt_r**\(3\)](#), [**exp**\(1\)](#), [**explain**\(3\)](#), [**explain_sprintf**\(3\)](#), [**explain_sprintf_or_die**\(3\)](#), [**fcvt**\(3\)](#), [**fwprintf**\(3\)](#), [**gcv**\(3\)](#), [**getkey**\(1\)](#), [**lber-encode**\(3\)](#), [**mawk**\(1\)](#), [**perlfunc**\(1\)](#), [**pth**\(3\)](#), [**qfcvt**\(3\)](#), [**slapd-config**\(5\)](#), [**slapd.conf**\(5\)](#), [**slappasswd**\(8\)](#), [**strfmon**\(3\)](#), [**strftime**\(3\)](#), [**strongswan_todata**\(3\)](#)

strcmp(3) - Linux man page

Name

strcmp, strncmp - compare two strings

Synopsis

```
#include <string.h>
int strcmp(const char *s1, const char *s2);
int strncmp(const char *s1, const char *s2, size_t n);
```

Description

The **strcmp()** function compares the two strings *s1* and *s2*. It returns an integer less than, equal to, or greater than zero if *s1* is found, respectively, to be less than, to match, or be greater than *s2*.

The **strncmp()** function is similar, except it only compares the first (at most) *n* bytes of *s1* and *s2*.

Return Value

The **strcmp()** and **strncmp()** functions return an integer less than, equal to, or greater than zero if *s1* (or the first *n* bytes thereof) is found, respectively, to be less than, to match, or be greater than *s2*.



Conforming to

SVr4, 4.3BSD, C89, C99.

See Also

[***bcmp\(3\)***](#), [***memcmp\(3\)***](#), [***strcasecmp\(3\)***](#), [***strcoll\(3\)***](#), [***string\(3\)***](#), [***strncasecmp\(3\)***](#), [***strverscmp\(3\)***](#), [***wscmp\(3\)***](#), [***wcsncmp\(3\)***](#)

Referenced By

[***curl_strequal\(3\)***](#), [***hsearch\(3\)***](#), [***librasqal\(3\)***](#), [***namealloc\(3\)***](#), [***naturalstrcmp\(3\)***](#), [***qsort\(3\)***](#), [***scandir\(3\)***](#), [***strxfrm\(3\)***](#)

strcpy(3) - Linux man page

Name

strcpy, strncpy - copy a string

Synopsis

```
#include <string.h>
```

```
char *strcpy(char *dest, const char *src);
```

```
char *strncpy(char *dest, const char *src, size_t n);
```

Description

The **strcpy()** function copies the string pointed to by *src*, including the terminating null byte ('\0'), to the buffer pointed to by *dest*. The strings may not overlap, and the destination string *dest* must be large enough to receive the copy. *Beware of buffer overruns!* (See BUGS.)

The **strncpy()** function is similar, except that at most *n* bytes of *src* are copied. **Warning:** If there is no null byte among the first *n* bytes of *src*, the string placed in *dest* will not null-terminated. 

If the length of *src* is less than *n*, **strncpy()** writes additional null bytes to *dest* to ensure that a total of *n* bytes are written.

A simple implementation of **strncpy()** might be:

```
char *
strncpy(char *dest, const char *src, size_t n)
{
    size_t i;

    for (i = 0; i < n && src[i] != '\0'; i++)
        dest[i] = src[i];
    for ( ; i < n; i++)
        dest[i] = '\0';

    return dest;
}
```

Return Value

The **strcpy()** and **strncpy()** functions return a pointer to the destination string *dest*.

Conforming To

SVr4, 4.3BSD, C89, C99.

Notes

Some programmers consider **strncpy()** to be inefficient and error prone. If the programmer knows (i.e., includes code to test!) that the size of *dest* is greater than the length of *src*, then **strcpy()** can be used.

One valid (and intended) use of **strncpy()** is to copy a C string to a fixed-length buffer while ensuring both that the buffer is not overflowed and that unused bytes in the target buffer are zeroed out (perhaps to prevent information leaks if the buffer is to be written to media or transmitted to another process via an interprocess communication technique).

If there is no terminating null byte in the first *n* bytes of *src*, **strncpy()** produces an unterminated string in *dest*. You can force termination using something like the following:

```
strncpy(buf, str, n);
if (n > 0)
    buf[n - 1] = '\0';
```

(Of course, the above technique ignores the fact that information contained in *src* is lost in the copying to *dest*.)



Some systems (the BSDs, Solaris, and others) provide the following function:

```
size_t strlcpy(char *dest, const char *src, size_t size);
```

This function is similar to **strncpy()**, but it copies at most *size-1* bytes to *dest*, always adds a terminating null byte, and does not pad the target with (further) null bytes. This function fixes some of the problems of **strcpy()** and **strncpy()**, but the caller must still handle the possibility of data loss if *size* is too small. The return value of the function is the length of *src*, which allows truncation to be easily detected: if the return value is greater than or equal to *size*, truncation occurred. If loss of data matters, the caller *must* either check the arguments before the call, or test the function return value. **strlcpy()** is not present in glibc and is not standardized by POSIX, but is available on Linux via the *libbsd* library.

Bugs

If the destination string of a **strcpy()** is not large enough, then anything might happen. Overflowing fixed-length string buffers is a favorite cracker technique for taking complete control of the machine. Any time a program reads or copies data into a buffer, the program first needs to check that there's enough space. This may be unnecessary if you can show that overflow is impossible, but be careful: programs can get changed over time, in ways that may make the impossible possible.

See Also

[bcopy\(3\)](#), [memccpy\(3\)](#), [memcpy\(3\)](#), [memmove\(3\)](#), [stpcpy\(3\)](#), [stpncpy\(3\)](#), [strdup\(3\)](#), [string\(3\)](#), [wscpy\(3\)](#), [wcsncpy\(3\)](#)

Referenced By

[db2x_manxml\(1\)](#), [feature_test_macros\(7\)](#), [fmt_str\(3\)](#), [ss_string_copy\(3\)](#), [strcat\(3\)](#), [wcpcpy\(3\)](#)

